

Department of Information Technology

University of Gujrat



GENETICS INSIGHT

Session: BSIT: Spring 2021-2025

Project Advisor: Mr. Muhammad Attique

Submitted By:

Ahtasham-ul-hassan 21014156-001

Syed Umer Zubair 21014156-016

Faria Mahboob 21014156-024

Department of Information Technology
University of Gujrat

STATEMENT OF SUBMISSION

This is certify that Ahtasham-ul-hassan, Roll No. 21014156-001, Syed Umer Zubair Roll No. 21014156-016, and Faria Mahboob, Roll No. 21014156-024 has successfully completed the final year project named as Genetic-insight at the Department of Information Technology, University of Gujrat, to fulfill the requirement of the degree of BS in Information Technology.



03/02/2025

Project Supervisor

Project Management Office,
Faculty of C& IT Evening-UOG

Coordinator Evening/HOD

Acknowledgement

We truly acknowledge the cooperation and help make by **Mr. Muhammad Attique**, our Project supervisor, assistant professor, C&IT Evening Program, University of Gujrat. He has been a constant source of guidance throughout the course of this project. We also extend our appreciation to **Mr. Junaid Muzaffar**, a member of the defense committee, for his valuable time, evaluation, and feedback. We are also deeply thankful to our friends and families, whose silent support and unwavering encouragement motivated us to complete this project successfully.

Date: January 30, 2025

Abstract

Genetic-Insight is an advanced web-based platform designed to simplify and automate the process of classifying proteins and genes using modern machine learning techniques. Tailored for researchers, bioinformaticians, and life science professionals, the system eliminates the need for programming knowledge by offering a fully interactive, no-code interface.

The platform supports end-to-end biological data analysis — from uploading datasets in various formats (FASTA, CSV, Excel) to feature extraction using descriptors like AAC, K-mer, and CKSAAP. It empowers users to build and train machine learning models (e.g., SVM, Random Forest, KNN), evaluate their performance through visual metrics such as confusion matrices and ROC curves, and generate accurate predictions on new data.

Genetic-Insight promotes accessibility, reproducibility, and speed in biological classification tasks. With built-in modules for dataset processing, feature engineering, model building, and result visualization, it streamlines complex workflows into a guided, user-friendly experience. The platform is ideal for academic research, institutional analysis, and experimental exploration in computational biology.

TABLE OF CONTENTS

CHAPTER 1: PROJECT FEASIBILITY REPORT	7
1.1. INTRODUCTION	7
1.2. PROJECT/PRODUCT FEASIBILITY REPORT	7
1.2.1. <i>Technical Feasibility</i>	1
1.2.2. <i>Operational Feasibility</i>	1
1.2.3. <i>Economic Feasibility</i>	1
1.2.4. <i>Schedule Feasibility</i>	1
1.2.5. <i>Specification Feasibility</i>	1
1.2.6. <i>Information Feasibility</i>	2
1.2.7. <i>Motivational Feasibility</i>	2
1.2.8. <i>Legal & Ethical Feasibility</i>	2
1.3. PROJECT/PRODUCT SCOPE	2
1.3.1 <i>Project Objectives</i>	3
1.4. PROJECT/PRODUCT COSTING	3
1.4.1. <i>Project Cost Estimation by Function Point Analysis</i>	4
1.4.2. <i>Project Cost Estimation by using COCOMO'81 (Constructive Cost Model)</i> ...	6
1.4.3. <i>Activity Based Costing</i>	8
1.4.4 <i>Estimated Cost Breakdown for Genetic-Insight</i>	9
1.5. TASK DEPENDENCY TABLE	10
1.6. CPM - CRITICAL PATH METHOD	12
1.7. GANTT CHART	15
1.8. INTRODUCTION TO TEAM MEMBER AND THEIR SKILL SET	16
1.10. TOOLS AND TECHNOLOGY WITH REASONING	17
1.11. VISION DOCUMENT	18
1.12. RISK LIST	20
1.13. PRODUCT FEATURES/ PRODUCT DECOMPOSITION	21
CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION (FOR OBJECT ORIENTED APPROACH)	25
2.1 INTRODUCTION:	25
2.1.1 <i>Systems Specifications</i>	25
2.1.2. <i>Identifying External Entities</i>	26
2.1.3. <i>Context Level Data Flow Diagram:</i>	29
2.2. EXISTING SYSTEM	29
2.2.1. <i>Scope of the System</i>	29
2.2.2. <i>Summary of Requirements: (Initial Requirements)</i>	30
2.2.3. <i>Identifying External Entities:</i>	31
2.2.4. <i>Capture "shall" Statements:</i>	33
2.2.5. <i>Allocate and Priorities Requirements:</i>	34
2.2.6. <i>Requirements Traceability Matrix:</i>	35
2.2.7. <i>High Level Usecase Diagram:</i>	37
2.2.8. <i>Usecase Description</i>	38

CHAPTER 3: DESIGN DOCUMENT (FOR OBJECT ORIENTED APPROACH)41

3.1. INTRODUCTION:41

3.2. DOMAIN MODEL	43
3.3. SYSTEM SEQUENCE DIAGRAM	44
3.4. SEQUENCE DIAGRAM	45
3.4.1. <i>Defining a Sequence diagram</i>	46
3.5. COLLABORATION DIAGRAM.....	48
3.5.1. <i>Contents of Collaboration Diagrams</i>	48
3.6. OPERATION CONTRACTS	49
3.7. DESIGN CLASS DIAGRAM.....	50
3.8. STATE CHART DIAGRAM	53
3.9. DATA MODEL.....	55

CHAPTER 4: USER INTERFACE DESIGN57

4.1. INTRODUCTION.....	57
4.2. SITE MAPS	57
4.3. STORY BOARDS	59
USER CLICKS ON “GET STARTED” TO BEGIN.....	62
SELECT THE DESCRIPTOR AND PARAMETERS:.....	64
4.4. NAVIGATIONAL MAPS:	69
NAVIGATIONAL FLOW DIAGRAM4.5 TRACE-ABILITY MATRIX	71

CHAPTER 5: SOFTWARE TESTING72

5.1 INTRODUCTION:	73
5.2 TEST PLAN	73
5.2.1 <i>Purpose</i>	73
5.2.2 <i>Outline</i>	73
5.3 TEST DESIGN SPECIFICATION	77
5.3.1 <i>Purpose</i>	77
5.3.2 <i>Outline</i>	77
5.4. TEST CASE SPECIFICATION.....	80
5.4.1. <i>Purpose</i>	80
5.4.2. <i>Outline</i>	80
5.5 TEST PROCEDURE SPECIFICATION	82
5.5.1 <i>Purpose</i>	82
5.5.2 <i>Outline</i>	82
5.6 TEST ITEM TRANSMITTAL REPORT.....	84
5.6.1 <i>Purpose</i>	84
5.6.2 <i>Outline</i>	84
5.7 TEST LOG	85
5.7.1 <i>Purpose</i>	85
5.7.2 <i>Outline</i>	85
5.8 TEST INCIDENT REPORT	87
5.8.1 <i>Purpose</i>	87
5.8.2 <i>Outline</i>	87
5.9 TEST SUMMARY REPORT.....	88
5.9.1 <i>Purpose</i>	88
5.9.2 <i>Outline</i>	89

Chapter 1:

Project Feasibility Report

1.1. Introduction

Genetic-Insight is a SaaS platform designed to assist researchers in genomics and proteomics by enabling the classification of genes and proteins using machine learning models. The platform provides a user-friendly interface where users can upload datasets, configure model settings, and train machine learning models without requiring coding expertise.

Genetic-Insight automates key processes such as feature engineering, data preprocessing, feature selection, dataset balancing, model implementation, cross-validation, and result visualization. By streamlining these complex tasks, the platform allows researchers to focus on data analysis and scientific discovery rather than the technical challenges of machine learning implementation. With its ability to handle large-scale biological data, Genetic-Insight aims to accelerate research in genomics and proteomics, making computational intelligence more accessible to the scientific community.

Genetic-Insight automates critical steps in the machine learning pipeline, including data preprocessing, feature engineering, dataset balancing, and model evaluation. The platform also integrates cross-validation techniques and advanced result visualization to enhance the accuracy and interpretability of model predictions. By streamlining these processes, Genetic-Insight ensures that researchers can focus on deriving valuable insights from their data, rather than being bogged down by technical intricacies.

Designed with the goal of accelerating research, Genetic-Insight serves as a powerful tool for genomic and proteomic studies. Its ease of use and computational intelligence enable researchers to conduct large-scale data analysis quickly and effectively. Whether identifying potential genetic markers or exploring protein structures, Genetic-Insight plays a vital role in advancing scientific understanding and fostering breakthroughs in personalized medicine and other research domains.

1.2. Project/Product Feasibility Report

The first step in any project is to assess its feasibility, ensuring it can be practically developed and beneficial to the target users or organization. Feasibility involves evaluating the availability of data, resources, time, and expertise needed to execute the project. It helps determine whether developing the software system is practical and worthwhile. Feasibility is an ongoing process, continuously reassessed throughout the project lifecycle. Below is the feasibility analysis for the Genetic-Insight project.

1. Technical Feasibility
2. Operational Feasibility
3. Economic Feasibility
4. Schedule Feasibility
5. Specification Feasibility
6. Information Feasibility
7. Motivational Feasibility
8. Legal and Ethical Feasibility

1.2.1. Technical Feasibility

Genetic-Insight is built using Python-based machine learning libraries such as TensorFlow and Scikit-learn, ensuring robust and scalable performance. The platform incorporates proven classification techniques and automated workflows, making it highly effective for genomic and proteomic research. Additionally, the development team has expertise in AI, ML, and software development, ensuring the successful implementation and maintenance of the system.

1.2.2. Operational Feasibility

The platform is designed with an intuitive and user-friendly interface that allows researchers to easily upload datasets, configure machine learning models, and analyze results. Automated processes, such as feature engineering, model selection, and evaluation, reduce the complexity of machine learning tasks, making the platform accessible to users with varying levels of expertise. Its seamless integration with existing research workflows enhances operational efficiency.

1.2.3. Economic Feasibility

Genetic-Insight operates on a **SaaS (Software-as-a-Service)** model with multiple subscription tiers, providing a sustainable revenue stream. The use of open-source technologies significantly reduces development and operational costs, while the increasing demand for AI-driven genomic research tools ensures a strong market potential and return on investment. The business model supports scalability without requiring heavy upfront investment.

1.2.4. Schedule Feasibility

The project is planned to be completed within **6 to 8 months**, following a structured development roadmap that includes research, prototyping, testing, and deployment phases. By breaking down the project into manageable milestones, potential risks can be mitigated, ensuring timely delivery without compromising quality.

1.2.5. Specification Feasibility

The platform meets essential functional requirements, including **dataset upload, preprocessing, model training, evaluation, and results visualization**. These capabilities align with the needs of researchers in genomics and proteomics, providing them with a streamlined workflow for analyzing biological data using AI-driven insights. The system is designed to handle large datasets efficiently, ensuring high performance.

1.2.6. Information Feasibility

The platform requires access to high-quality genomic and proteomic datasets, which are available from **publicly accessible databases and research collaborations**. The availability of such datasets ensures that the platform has sufficient real-world data for model training and validation, supporting the effectiveness of its predictions and insights.

1.2.7. Motivational Feasibility

The development team is highly motivated by the platform's potential to **revolutionize genomic research** by simplifying machine learning applications for scientists. The opportunity to contribute to advancements in biotechnology and healthcare fuels continuous innovation and commitment to the project. The expected impact on scientific discovery and personalized medicine further drives enthusiasm for the project.

1.2.8. Legal & Ethical Feasibility

Genetic-Insight complies with **data privacy regulations** such as **GDPR** and follows ethical guidelines for handling sensitive genetic data. The platform ensures data security through encryption, controlled access, and anonymization techniques where necessary. Compliance with ethical standards in genomic research prevents misuse of genetic data, ensuring that the system operates within a legally and ethically responsible framework.

Conclusion

Genetic-Insight is feasible across all dimensions—**technical, operational, economic, schedule, specification, information, motivational, and legal**. The project leverages cutting-edge technology, aligns with user needs, ensures financial sustainability, and adheres to legal and ethical standards. With a structured development plan and a strong market demand, Genetic-Insight is well-positioned to succeed as an AI-powered tool for advancing genomic research.

1.3. Project/Product Scope

Genetic-Insight is a **SaaS-based AI platform** designed to assist researchers in genomics and proteomics by providing automated machine learning tools for **classification, feature selection, and model evaluation**. The platform simplifies complex bioinformatics tasks, enabling users to analyze genetic and protein datasets efficiently without requiring advanced programming knowledge.

Core Functionalities

- Secure dataset upload and storage.
- Automated feature engineering, normalization, and dataset balancing.
- Support for multiple classification algorithms such as Random Forest, SVM, and XGBoost.
- Customizable hyperparameter tuning and cross-validation techniques.
- Interactive result visualization, including confusion matrix and ROC curves.
- Feature importance analysis for identifying significant genetic markers.
- Comparative analysis of different models for performance evaluation.

Technology Stack:

The platform will be developed as a **web-based SaaS application**, ensuring accessibility

and scalability. Key technologies include:

- **Backend:** Python (Flask/Django) for machine learning processing.
- **Frontend:** HTML, CSS / JavaScript (React) for an intuitive user interface.

1.3.1 Project Objectives

- Provide an **AI-powered platform** that simplifies genetic and proteomic research.
- Enable researchers to **train and test models** without writing complex code.
- Offer a **user-friendly, interactive environment** for data analysis.
- Ensure **data security and compliance** with regulations like GDPR.

1.4. Project/Product Costing

Cost estimation for **Genetic-Insight** involves a structured approach using various metrics to ensure accurate predictions and effective resource management. These metrics are categorized as follows:

- **Knowledge-Oriented Metrics:** These track the development process to evaluate, predict, or monitor progress. For Genetic-Insight, this includes tracking the time required for tasks like data preprocessing, feature engineering, and AI model training.
- **Achievement-Oriented Metrics:** These focus on measuring product attributes, such as system performance, model accuracy, user satisfaction, and system scalability.

Algorithmic Cost Modeling

For **Genetic-Insight**, cost estimation relies on **algorithmic cost modeling**. This process involves:

- Using historical data and mathematical formulas to link inputs, such as team size and complexity of model implementation, with estimated outputs, like development timelines and costs.
- Calibrating models to suit **Genetic-Insight's** specific development environment by adjusting the weightings for various metrics based on unique project factors.

Key Cost Components

- **Development Costs:** Include salaries for developers, costs for tools like Python, PHP, machine learning libraries (e.g., TensorFlow, OpenCV), and resources for implementing modules for data classification and visualization.
- **Operational Costs:** Cover cloud hosting, ongoing maintenance, model retraining, and customer support for the web-based platform.
- **Marketing and Launch Costs:** Focus on promoting **Genetic-Insight** to reach target audiences in the scientific community and drive user adoption.

Enhancing Cost Accuracy

Cost accuracy for **Genetic-Insight** can be improved by:

- Breaking down the project into smaller tasks, such as developing individual machine learning models, testing algorithms, and building the frontend interface.
- Using lifecycle cost models to account for both development and long-term

maintenance costs.

- Regular reviews and adjustments to cost estimates as the project progresses.

By leveraging these methods, **Genetic-Insight** ensures that financial resources are allocated efficiently, enabling successful project delivery within budget constraints.

1.4.1. Project Cost Estimation by Function Point Analysis

Function-oriented software metrics, like those used for **Genetic-Insight**, measure the functionality delivered by the system. Since functionality cannot be directly measured, it is derived using direct metrics based on the application's information domain and complexity assessments.

Function Point Analysis (FPA) is employed to track and monitor scope changes throughout the project phases—from requirements gathering to implementation. By comparing function points at various stages, **Genetic-Insight** can identify scope creep and improve communication with stakeholders over time.

The process involves the following:

Based on the description of "Genetic-Insight", we will estimate the following counts for the information domain characteristics:

Let's assume the following hypothetical counts for **Genetic-Insight**:

- **User Inputs (UI):**
(e.g., uploading datasets, configuring model settings, etc.)
- **User Outputs (UO):**
(e.g., model predictions, evaluation results, etc.)
- **User Inquiries (UIQ):**
(e.g., querying model performance, dataset status, etc.)
- **Files (F):**
(e.g., datasets, model configurations, result files, etc.)
- **External Interfaces (EI):**
(e.g., APIs for dataset retrieval, external databases, etc.)

Estimate Counts

Let's assume the following hypothetical counts for "**Genetic-Insight**":

- User Inputs (UI): 10
- User Outputs (UO): 8
- User Inquiries (UIQ): 6
- Files (F):
- External Interfaces (EI): 4

Assign Complexity Values

Assigning complexity values (simple, average, complex) to each count. For **Genetic-Insight**, the complexity values are:

- **User Inputs (UI):** Average (4)

- **User Outputs (UO):** Average (4)
- **User Inquiries (UIQ):** Average (4)
- **Files (F):** Average (4)
- **External Interfaces (EI):** Complex (5)

1. Calculate Function Points (FP)

Using the formula:

$$FP_{est} = \text{Count Total} \times [0.65 + 0.01 \times (Fi)]$$

Step 1: Calculate Count Total

Count Total = (UI * UI Complexity) + (UO * UO Complexity) + (UIQ * UIQ Complexity) + (F * F Complexity) + (EI * EI Complexity)

$$\text{Count Total} = (10 \times 4) + (8 \times 4) + (6 \times 4) + (5 \times 4) + (4 \times 5)$$

$$\text{Count Total} = 40 + 32 + 24 + 20 + 20 = 136$$

Step 2: Calculate Value Adjustment Factor (VAF)

Assuming an average score of 2 for the 14 general system characteristics (GSCs):

$$Fi = \text{Sum of GSC scores} = 2 \times 14 = 28$$

Step 3: Calculate FP_{est}.

$$FP_{est.} = 136 \times [0.65 + 0.01 \times 28]$$

$$FP_{est.} = 136 \times [0.65 + 0.28]$$

$$FP_{est.} = 136 \times 0.93$$

$$FP_{est.} = 126.48 \text{ (approximately 126 FP)}$$

Step 4: Calculate Total Project Cost and Total Project Effort

Assuming the following values for calculations:

- **Labor Rate:** \$50/hour
- **Productivity Parameter:** 10 FP/person/day

Step 5: Calculate Cost per FP

$$\text{Cost per FP} = \text{Labor Rate} / \text{Productivity Parameter} = 50 / 10 = 5 \text{ (dollars per FP)}$$

Step 6: Calculate Total Project Cost

$$\text{Total Project Cost} = FP_{est.} \times (\text{Cost per FP})$$

$$\text{Total Project Cost} = 126 \times 5\$ = \$630 \text{ (dollars)}$$

Step 7: Calculate Total Estimated Effort

Total Estimated Effort = $FP_{est.} / \text{Productivity Parameter}$

Total Estimated Effort = $126/10 = 12.6$ (approximately 13 days)

Summary of Functional Point Analysis for Genetic-Insight

- **Function Points (FP est.):** 126
- **Total Project Cost:** \$630
- **Total Estimated Effort:** 13 days

This structured approach provides a clear estimation of the project cost and effort based on **Function Point Analysis** for your **Genetic-Insight** project. Adjust the counts and complexity values as necessary based on your specific project details for more accurate results.

1.4.2. Project Cost Estimation by using COCOMO'81 (Constructive Cost Model)

COCOMO Model Overview

COCOMO classifies software projects into three categories based on their complexity and the development environment:

1. **Organic:** Projects developed by small teams with good experience in the application area and the software being developed.
2. **Semi-Detached:** Projects that are intermediate in size and complexity, requiring a mix of experienced and less experienced personnel.
3. **Embedded:** Projects that are complex, with tight hardware and operational constraints, requiring highly specialized personnel.

To estimate the effort, schedule, and other parameters for your **Genetic-Insight** project using the **COCOMO** model, we'll follow a structured approach:

1. Estimating the Lines of Code (LOC) for Genetic-Insight

Given the LOC of similar projects:

- **Project 1:** 9,000 LOC
- **Project 2:** 11,300 LOC
- **Project 3:** 10,400 LOC

The average LOC is calculated as: 10,233 LOC.

$(9000 + 11300 + 10400) / 3 = 10,233$

Therefore, the estimated size for **Genetic-Insight** is approximately **10,233 LOC OR 10.2 KLOC**.

2. Selecting the Appropriate COCOMO Model

COCOMO classifies projects into three categories:

- **Organic:** Small, simple projects developed by small teams with good experience.
- **Semi-Detached:** Intermediate projects with a mix of experienced and less

experienced personnel.

- **Embedded:** Complex projects with tight hardware and operational constraints.

Given the size of **Genetic-Insight** (10,233 LOC), "Genetic Insight" is a moderately complex project with a medium-sized team, the **Semi-Detached** category is appropriate.

3. Calculating Effort and Schedule Using Detailed COCOMO

For the **Semi-Detached** category, the Detailed COCOMO model provides the following formulas:

- **Effort (PM):** $PM = 3.0 \times (KLOC)^{1.12} \times EAF$
- **Development Time (TD):** $TD = 2.5 \times (PM)^{0.35}$

Calculating:

Step 1: Define Given Values

- **KLOC (Size of Project)** = 10.2
- **Effort Adjustment Factor (EAF)** = Assuming **1.0** (default if no cost drivers are given)
- **Detailed COCOMO Constants** (for Semi-Detached Model):
 - **ai = 3.0, bi = 1.12** (Effort equation constants)
 - **ci = 2.5, di = 0.35** (Duration equation constants)
 - **μp = 0.38** (Effort distribution constant)
 - **θp = 0.50** (Duration distribution constant)

Step 2: Calculate Effort (E)

The formula is:

$$E = ai \times (KLOC)^{bi} \times EAF$$

Substituting values:

$$E = 3.0 \times (10.2)^{1.12} \times 1.0$$

$$E = 3.0 \times 14.63$$

$$E = 43.89 \text{ person-months}$$

Step 3: Calculate Development Time (D)

Using:

$$D = ci \times (E)^{di}$$

$$D = 2.5 \times (43.89)^{0.35}$$

$$D = 2.5 \times 4.63$$

$$D = 11.57 \text{ months}$$

Step 4: Calculate Effort Distribution (Ep)

$$Ep = \mu p \times E$$

$$Ep = 0.38 \times 43.89$$

$$Ep = 16.68 \text{ person-months}$$

Step 5: Calculate Duration Distribution (Dp)

$$Dp = \theta p \times D$$

$$Dp = 0.50 \times 11.57$$

$$Dp = 5.79 \text{ months}$$

Step 6: Calculate Staff Size (SS)

$$SS = E/D$$

$$SS = 43.89/11.57$$

$$SS = 3.79 \approx 4 \text{ persons}$$

Step 7: Calculate Productivity (P)

$$P = KLOC/E$$

$$P = 10.2/43.89$$

$$P = 0.23 \text{ KLOC/person-month}$$

Final Results Summary

Metric	Value
Effort (E)	43.89 person-months
Development Time (D)	11.57 months
Effort Distribution (Ep)	16.68 person-months
Duration Distribution (Dp)	5.79 months
Staff Size (SS)	4 persons
Productivity (P)	0.23 KLOC/person-month

Table 1

This detailed COCOMO estimation provides an accurate projection for **Genetic-Insight**, ensuring efficient resource planning

1.4.3. Activity Based Costing

Activity-Based Costing (ABC) is a method used to estimate the cost of a project by assigning costs to specific activities rather than using traditional cost estimation methods. In the **Genetic-Insight** project, ABC helps allocate resources efficiently by identifying cost drivers associated with various software development activities.

Steps in Activity-Based Costing for Genetic-Insight

1. Identify Key Activities

The major activities in the Genetic-Insight project include:

- Requirement Analysis
- Data Preprocessing & Feature Engineering
- Model Development & Training
- Frontend & Backend Development
- System Integration & Testing
- Deployment & Maintenance

2. Assign Cost Drivers

Each activity has a cost driver that directly influences the total cost. The cost drivers for the Genetic-Insight project include:

- Labor Cost (developer hours, ML engineers, testers, etc.)
- Computational Resources (server costs, cloud storage, GPUs for training)
- Software Licenses (Python libraries, frameworks, API services)
- Infrastructure (hosting, database management, security protocols)

3. Calculate Cost Per Activity

The total cost for each activity is estimated based on the following formula:

$$\text{Total Cost of Activity} = \text{Cost Driver Rate} \times \text{Activity Resource Usage}$$

Example:

If the **Model Development** phase requires 3 ML engineers working for 2 months at a rate of **\$5,000 per month**, the cost would be:

$$\text{Total Cost} = 3 \times 2 \times 5000 = 30,000$$

4. Summarize Total Project Cost

The sum of all individual activity costs provides the estimated total project cost.

This method ensures accurate cost estimation and helps in budget allocation.

1.4.4 Estimated Cost Breakdown for Genetic-Insight

Activity	Estimated Cost (USD)
Requirement Analysis	\$10,000
Data Preprocessing & Feature Engineering	\$12,000
Machine Learning Model Development	\$30,000
Frontend & Backend Development	\$25,000
System Integration & Testing	\$15,000
Deployment & Maintenance	\$8,000
Total Estimated Cost	\$100,000

Table 2

Conclusion

The **Activity-Based Costing (ABC) method** provides a structured and accurate estimation of project expenses by breaking down costs based on specific activities. This ensures efficient resource allocation and helps in financial planning for the **Genetic-Insight** project.

Basic Cost Drivers:

- In **Activity-Based Costing (ABC)**, cost drivers are the factors that influence the cost of various activities. The **Genetic-Insight** project involves several key cost drivers that impact the total cost estimation

Basic Cost Drivers in Genetic-Insight

1. Personnel Costs (Labor Cost)

- Software Developers (Frontend & Backend)
- Machine Learning Engineers (Model Training & Optimization)
- Data Scientists (Preprocessing & Feature Engineering)
- Quality Assurance & Testers (Testing & Debugging)

- Project Management & Documentation

Impact: Labor costs are determined by the hourly rate and total working hours required per phase.

2. Hardware & Computational Costs

- High-Performance Servers & GPUs (for model training)
- Cloud Storage & Computing Services (AWS, Google Cloud, Azure)
- Development & Testing Machines

Impact: The cost is driven by the duration of model training, required computational power, and data storage needs.

3. Software & Licensing Costs

- Programming Frameworks & Libraries (Python, TensorFlow, Scikit-Learn, etc.)
- Development & Deployment Tools (Docker, Kubernetes, CI/CD tools)
- Database Services (PostgreSQL, MongoDB, MySQL)

Impact: Some tools and libraries require paid licenses or subscriptions, which add to project costs.

4. Infrastructure & Hosting Costs

- Web Hosting & Domain Registration
- Database Management Systems
- Security & Compliance Tools

Impact: Ongoing costs for hosting, maintenance, and data security.

5. Testing & Quality Assurance Costs

- Automated Testing Tools
- Security Audits
- Usability Testing & User Feedback Implementation

Impact: Ensures software reliability and functionality before deployment.

6. Deployment & Maintenance Costs

- Server Setup & Configuration
- System Updates & Bug Fixes
- Customer Support & Maintenance

Impact: Required for long-term project sustainability and continuous improvements.

1.5. Task Dependency Table

Following the steps outlined, the **Task Dependency Table** for **Genetic-Insight** is developed as follows:

Step 1: Brainstorming the Tasks

The project tasks are identified based on the **Genetic-Insight** project workflow:

1. Requirements Gathering
2. System Architecture Design
3. Data Collection & Preprocessing
4. Feature Engineering
5. Model Selection & Implementation
6. Hyperparameter Tuning
7. Backend Development (API)
8. Frontend Development (UI/UX)
9. Integration (Backend + Frontend)
10. Model Integration with Backend
11. Testing (Unit & System)
12. Deployment (Cloud & Server)
13. Performance Monitoring
14. Maintenance & Updates

Step 2: Organizing the Tasks on Job Cards

Each of these tasks is written on a **Post-it Note** (or represented in a list).

Step 3: Establishing Dependencies & Arranging in Sequence and Parallel

The table below represents the dependencies:

Task ID	Task Name	Dependent On	Type of Dependency
T1	Requirements Gathering	-	-
T2	System Architecture Design	T1	Finish-to-Start (FS)
T3	Data Collection & Preprocessing	T1	Finish-to-Start (FS)
T4	Feature Engineering	T3	Finish-to-Start (FS)
T5	Model Selection & Implementation	T2, T4	Finish-to-Start (FS)
T6	Hyperparameter Tuning	T5	Finish-to-Start (FS)
T7	Backend Development (API)	T2	Finish-to-Start (FS)
T8	Frontend Development (UI/UX)	T2	Finish-to-Start (FS)
T9	Integration (Backend + Frontend)	T7, T8	Finish-to-Start (FS)
T10	Model Integration with Backend	T6, T7	Finish-to-Start (FS)
T11	Testing (Unit & System)	T9, T10	Finish-to-Start (FS)
T12	Deployment (Cloud & Server)	T11	Finish-to-Start (FS)
T13	Performance Monitoring	T12	Start-to-Start (SS)
T14	Maintenance & Updates	T12	Start-to-Start (SS)

Table 3

Dependency Types

- **Finish-to-Start (FS):** Task B cannot start until Task A is completed.
- **Start-to-Start (SS):** Task B starts only after Task A has started.

This **Task Dependency Table** ensures a logical flow in the project, preventing bottlenecks and optimizing resource allocation

1.6. CPM - Critical Path Method

Steps in CPM Project Planning

1. Specify the individual activities.
2. Determine the sequence of those activities.
3. Draw a network diagram.
4. Estimate the completion time for each activity.
5. Identify the critical path (longest path through the network).
6. Update the CPM diagram as the project progresses.

Project Activities and Dependencies

Activity	Description	Duration (Days)	Start Date	End Date	Immediate Predecessors
A	Requirements Gathering	20	31-Jan-25	27-Feb-25	None
B	System Architecture Design	10	28-Feb-25	13-Mar-25	A
C	Data Collection & Preprocessing	6	28-Feb-25	7-Mar-25	A
D	Feature Engineering	5	10-Mar-25	14-Mar-25	C
E	Model Selection & Implementation	3	17-Mar-25	19-Mar-25	B, D
F	Hyperparameter Tuning	3	20-Mar-25	24-Mar-25	E
G	Backend Development (API)	10	14-Mar-25	27-Mar-25	B
H	Frontend Development (UI/UX)	10	14-Mar-25	27-Mar-25	B
I	Integration (Backend + Frontend)	7	28-Mar-25	7-Apr-25	G, H
J	Model Integration with Backend	4	28-Mar-25	2-Apr-25	F, G
K	Testing (Unit & System)	2	8-Apr-25	9-Apr-25	I, J
L	Deployment (Cloud & Server)	2	10-Apr-25	11-Apr-25	K
M	Performance Monitoring	1	14-Apr-25	14-Apr-25	L
N	Maintenance & Updates	1	14-Apr-25	14-Apr-25	L

Table 4

1.7. Draw the Network Diagram

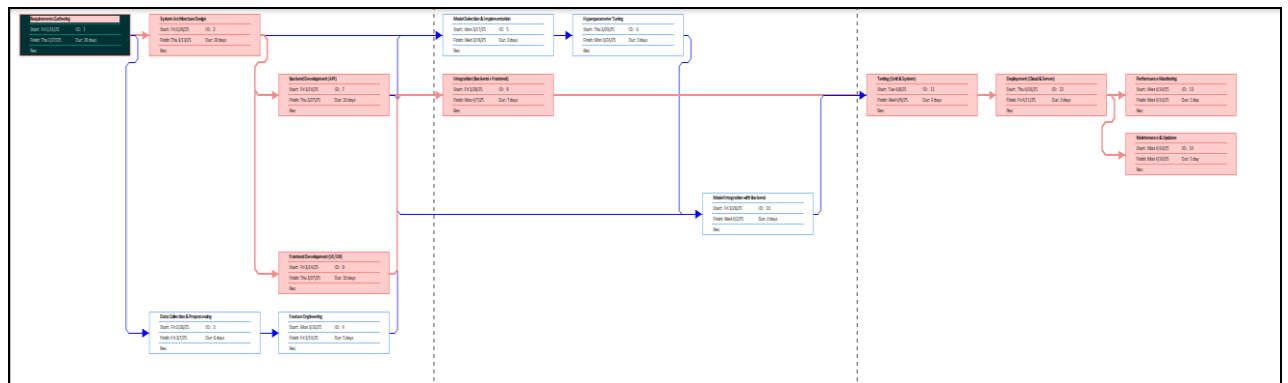


Figure 1

1.8. Estimate Activity Completion Time

For **Genetic-Inspight** the estimated activity completion times are as follows:

Activity	Duration (Days)
Requirements Gathering	20
System Architecture Design	10
Data Collection & Preprocessing	6
Feature Engineering	5
Model Selection & Implementation	3
Hyperparameter Tuning	3
Backend Development (API)	10
Frontend Development (UI/UX)	10
Integration (Backend + Frontend)	7
Model Integration with Backend	4
Testing (Unit & System)	2
Deployment (Cloud & Server)	2
Performance Monitoring	1
Maintenance & Updates	1

This estimation is based on the **deterministic** approach of **CPM**, meaning each task has a fixed time duration without variability

1.9. Identify the Critical Path

Critical Path Method (CPM) Analysis for Genetic-Inspight

1. Critical Path Activities

The critical path consists of the following activities, meaning any delay in these tasks will directly delay the project:

$A \rightarrow B \rightarrow G \rightarrow H \rightarrow I \rightarrow K \rightarrow L \rightarrow M \rightarrow N$

2. Activity Timeline

Each activity's **Earliest Start (ES)**, **Earliest Finish (EF)**, **Latest Start (LS)**, **Latest**

Finish (LF), and **Total Slack (TS)** are as follows:

3. Interpretation

- The **total project duration** is **52 days**, based on the longest path (Critical Path).
- Any delay in activities on the **Critical Path** will extend the total project completion time.
- Some activities (**C, D, E, F, J**) have slack time, meaning they can be delayed slightly without affecting the final deadline.

4. Next Steps

- **Prioritize** the activities on the critical path to ensure timely completion.
- **Monitor dependencies** closely, especially for Backend, Frontend, and Integration phases.
- **Plan contingencies** for activities with zero slack to avoid delays.

Activity	Duration (Days)	ES	EF	LS	LF	TS	FS
Requirements Gathering	20	0	20	0	20	0	0
System Architecture Design	10	20	30	20	30	0	0
Data Collection & Preprocessing	6	20	26	24	30	4	4
Feature Engineering	5	26	31	30	35	4	4
Model Selection & Implementation	3	30	33	35	38	5	2
Hyperparameter Tuning	3	33	36	38	41	5	5
Backend Development (API)	10	30	40	30	40	0	0
Frontend Development (UI/UX)	10	30	40	30	40	0	0
Integration (Backend + Frontend)	7	40	47	40	47	0	0
Model Integration with Backend	4	40	44	43	47	3	3
Testing (Unit & System)	2	47	49	47	49	0	0
Deployment (Cloud & Server)	2	49	51	49	51	0	0
Performance Monitoring	1	51	52	51	52	0	0
Maintenance & Updates	1	51	52	51	52	0	0

Table 5

Critical Path:

Requirements Gathering → System Architecture Design → Backend Development → Integration → Testing → Deployment → Performance Monitoring

Since all activities on the **critical path** have **zero slack (TS = 0)**, any delay in these tasks will delay the entire project.

1.7. Gantt chart

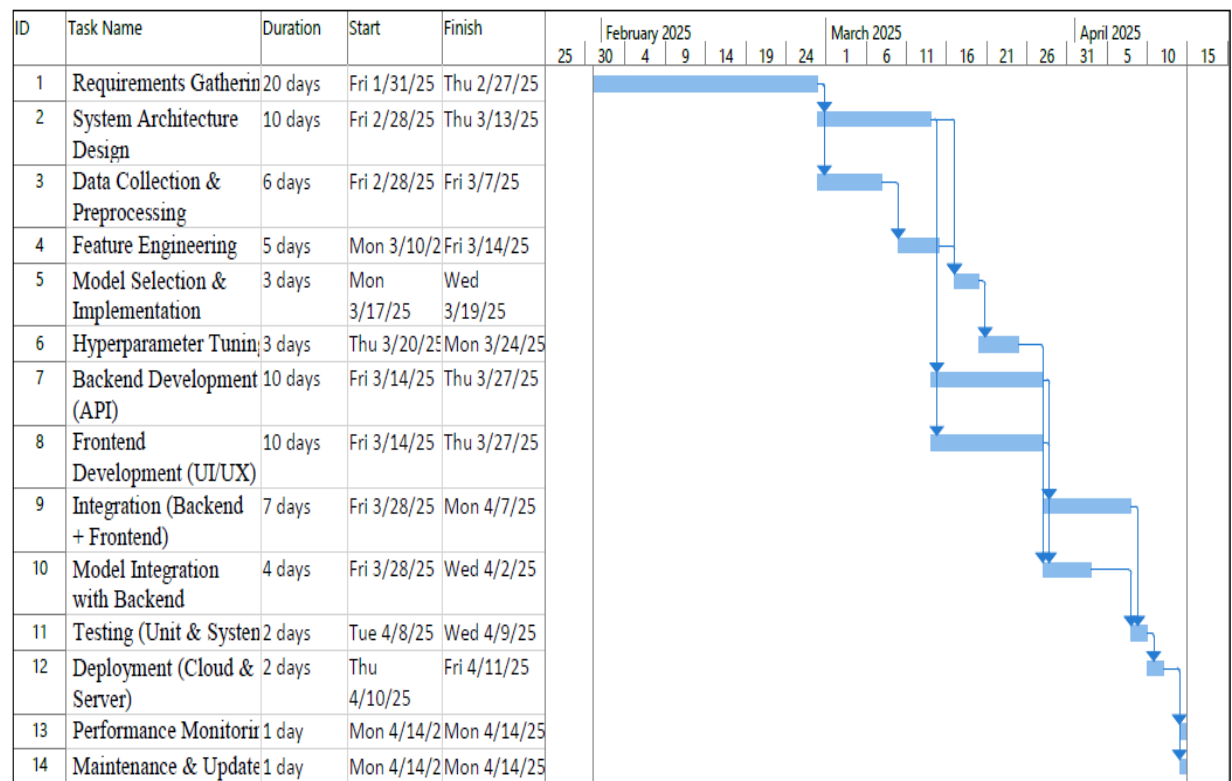


Figure 2

1.8. Introduction to Team member and their skill set

Activity	Duration (Days)	Members	Dependencies
Requirements Gathering	20	Umer (M1), Ahtasham (M2), Faria (M3)	Umer
System Architecture Design	10	Umer (M1), Ahtasham (M2), Faria (M3)	Umer, Faria
Data Collection & Preprocessing	6	Umer (M1), Ahtasham (M2), Faria (M3)	Umer, Faria, Ahtasham
Feature Engineering	5	Umer (M1), Ahtasham (M2), Faria (M3)	Umer
Model Selection & Implementation	3	Umer (M1), Ahtasham (M2), Faria (M3)	Umer
Hyperparameter Tuning	3	Umer (M1), Ahtasham (M2), Faria (M3)	Umer
Backend Development (API)	10	Umer (M1), Ahtasham (M2), Faria (M3)	Umer
Frontend Development (UI/UX)	10	Umer (M1), Ahtasham (M2), Faria (M3)	Faria, Ahtasham
Integration (Backend + Frontend)	7	Umer (M1), Ahtasham (M2), Faria (M3)	Faria, Ahtasham
Model Integration with Backend	4	Umer (M1), Ahtasham (M2), Faria (M3)	Umer, Ahtasham
Testing (Unit & System)	2	Umer (M1), Ahtasham (M2), Faria (M3)	Ahtasham
Deployment (Cloud & Server)	2	Umer (M1), Ahtasham (M2), Faria (M3)	All
Performance Monitoring	1	Umer (M1), Ahtasham (M2), Faria (M3)	Faria
Maintenance & Updates	1	Umer (M1), Ahtasham (M2), Faria (M3)	All
Final Report and Documentation	10	Umer (M1), Ahtasham (M2), Faria (M3)	All

Table 6

1.10. Tools and Technology with reasoning

Tool/Technology	Purpose/Reasoning
Python (3.12)	Used for backend development, Python is a versatile language for implementing machine learning models, data preprocessing, and system architecture. Python's libraries (e.g., scikit-learn, pandas, NumPy) are ideal for building machine learning solutions.
PHP	Chosen for frontend development as it is well-suited for web applications and integrates smoothly with HTML/CSS/JavaScript for dynamic content display and user interaction.
Flask/Django	Lightweight web frameworks for Python, used for building the web application. Flask is ideal for smaller projects, while Django is preferred for larger-scale applications.
scikit-learn	A key library for machine learning, offering tools for data preprocessing, model training, and evaluation. It supports a wide variety of algorithms, including Random Forest, SVM, and KNN.
XGBoost / CatBoost	These are high-performance machine learning models for classification tasks. XGBoost is known for its speed and performance, making it suitable for the project's demand for fast and accurate predictions.
TensorFlow / Keras	TensorFlow (and Keras as its high-level API) is used for implementing deep learning models, particularly neural networks, that may be applied to complex datasets.
Jupyter Notebooks	For prototype testing and model experimentation. It's easy to iterate on models and visualize results quickly.
Bootstrap	A framework for responsive frontend development. It speeds up the creation of clean, professional user interfaces without writing much custom CSS.
PyTorch	Used for deep learning applications where flexibility and speed are crucial. It can also be integrated with models for advanced AI research.

Table 7

These tools have been selected to offer robust, scalable, and maintainable solutions for both the backend and frontend of the Genetic-Insight platform, while also providing the necessary libraries for machine learning model development and deployment.

1.11. Vision Document

The **Vision Document** outlines the core goals, purpose, and overall direction for the Genetic-Insight project, providing a roadmap for its development and guiding its strategic decisions.

Project Vision

Genetic-Insight is a state-of-the-art Software as a Service (SaaS) platform designed to empower researchers, scientists, and the broader scientific community in the fields of genetics, bioinformatics, and molecular biology. The platform provides a web-based solution for classifying proteins, genes, and other genomic elements using advanced machine learning algorithms, without requiring any programming expertise. Users can upload datasets, configure model settings, and generate high-accuracy predictions, all within an intuitive and user-friendly interface.

Key Goals

1. **Automation and Simplification:** To automate complex tasks like feature engineering, model training, testing, and evaluation, allowing users to focus on scientific insights rather than technical details.
2. **User Empowerment:** Provide a user-friendly platform for researchers of varying skill levels to analyze genomic data, facilitating breakthroughs in genetic research.
3. **Flexibility and Scalability:** Enable users to upload custom datasets, select appropriate models, and customize module settings based on their specific research needs.
4. **Accurate and Reliable Results:** Ensure that the platform uses the most advanced machine learning models (Random Forest, SVM, Neural Networks, KNN) for high prediction accuracy, backed by rigorous cross-validation techniques.
5. **Visualization:** Provide interactive data visualizations that help users understand model performance, feature importance, and prediction results.
6. **Collaboration and Sharing:** Foster a collaborative research environment by allowing users to share datasets, models, and results with others in the scientific community.

Target Audience

- **Researchers and Scientists** in genomics, bioinformatics, and related fields.
- **Healthcare Professionals** using genetic data for diagnostics and personalized medicine.
- **Educators and Students** involved in genetic research and bioinformatics learning.
- **Biotech Companies** looking for a cost-effective and user-friendly platform to accelerate their research.

Core Features

- **Model Training and Evaluation:** Allows users to train machine learning models on genomic datasets and evaluate their performance using various metrics.
- **Dataset Upload and Management:** Users can upload, manage, and preprocess datasets for analysis.
- **Feature Engineering and Normalization:** Automatically processes data, handling missing values and optimizing feature selection.
- **Cross-validation and Auto-optimization:** Supports model evaluation with K-

fold cross-validation and parameter auto-optimization.

- **Results Visualization:** Provides graphical representations of results for better understanding and interpretation.
- **Model Export:** Users can save, load, and share their trained models for future use or predictions.

Strategic Vision

The Genetic-Insight platform aims to become the go-to tool for genomic data analysis, making cutting-edge machine learning accessible to the scientific community. By simplifying complex machine learning workflows, the platform will accelerate research in genetics and genomics, contributing to medical advancements, personalized healthcare, and a deeper understanding of the genetic basis of diseases.

Future Vision

- **Advanced AI Integration:** Incorporating deep learning models for complex genomic data.
- **Expanded Data Support:** Support for a wider range of genomic data types (e.g., RNA-Seq, whole-genome sequencing).
- **Cloud Integration:** Offering cloud-based computing for larger-scale data processing and model training.
- **Global Collaboration:** Facilitating international collaborations by allowing shared access to datasets, models, and results.

Conclusion

Genetic-Insight is an innovative platform designed to bridge the gap between cutting-edge machine learning technology and the genomic research community. By empowering users with advanced tools and a simple interface, Genetic-Insight aims to catalyze the future of genetic research and contribute to groundbreaking discoveries in molecular biology and medicine.

1.12. Risk List

The **Risk List** identifies potential risks that could impact the successful development and deployment of the Genetic-Insight project, along with strategies for mitigating these risks.

Risk	Impact	Probability	Mitigation Strategy
Data Privacy and Security Concerns	Sensitive data may be exposed or misused, causing legal and reputational damage.	Medium	Implement robust encryption and authentication measures for data storage and transmission. Comply with data protection regulations (e.g., GDPR).
Model Accuracy and Performance	Machine learning models may not meet accuracy expectations, impacting user trust.	High	Conduct thorough testing and cross-validation, fine-tune models, and consider using ensemble methods for better accuracy.
Scalability Issues	The platform may struggle to handle large datasets or high traffic.	Medium	Use scalable cloud infrastructure (e.g., AWS, Google Cloud) to ensure the platform can scale as needed. Leverage caching and load balancing.
Data Quality and Integrity	Inaccurate or incomplete datasets may lead to unreliable predictions.	Medium	Implement data validation and preprocessing techniques to clean and standardize datasets before analysis.
User Interface Usability Issues	Users may find the interface confusing or difficult to use, leading to lower adoption.	Medium	Conduct user testing and gather feedback to continuously improve the interface. Provide comprehensive user guides and tutorials.
Integration with External Data Sources	Difficulty integrating with diverse external data sources, such as various genomic databases.	Low	Build adaptable integration modules that support various data formats and APIs. Use standardized data formats (e.g., CSV, JSON) for seamless integration.
Regulatory and Compliance Risks	The platform may not comply with industry standards or regulations in different regions.	Medium	Stay updated on regulations related to genetic data (e.g., HIPAA, GDPR) and implement necessary compliance measures.

Lack of Expertise in ML/AI Implementation	The development team may lack deep expertise in machine learning, impacting model effectiveness.	Medium	Provide training to the development team on machine learning best practices. Consider collaborating with domain experts or hiring specialists.
Platform Downtime or Service Interruptions	Technical failures or service interruptions could disrupt user access to the platform.	Medium	Implement a robust monitoring and alerting system. Ensure backup servers and disaster recovery plans are in place to minimize downtime.
Budget Overruns	The project may exceed the allocated budget due to unforeseen costs or delays.	Medium	Establish a clear budget plan with contingencies for unexpected expenses. Monitor spending and timelines closely, adjusting resources as needed.
Delays in Development Timeline	The project may fall behind schedule due to unforeseen technical challenges or resource constraints.	High	Break the project into manageable milestones and regularly review progress. Allocate extra resources and prioritize critical tasks to stay on schedule.
Competition and Market Adoption	Competing platforms may emerge, or the platform may struggle to gain market adoption.	Medium	Differentiate the platform by offering unique features (e.g., ease of use, scalability) and ensure consistent marketing efforts to attract and retain users.
Technical Debt	Accumulation of quick, short-term solutions may lead to long-term maintenance challenges.	Medium	Prioritize code quality, refactoring, and documentation throughout the development process. Plan for regular code reviews and maintain a modular architecture.
Failure to Meet User Expectations	Users may expect more features or capabilities than what the platform can deliver at launch.	Medium	Set realistic expectations through clear communication with users. Release features incrementally, gathering feedback to refine and improve the platform over time.

Table 8

1.13. Product Features/ Product Decomposition

The **Product Features** and **Product Decomposition** outline the core features of the Genetic-Insight platform and break them down into smaller, manageable components for development and implementation.

Product Features

1. Dataset Upload and Management

- Allows users to upload custom datasets (e.g., protein, gene, and genomic data).
- Supports multiple file formats (CSV, Excel, JSON).
- Provides data validation and preprocessing to ensure clean, usable data.

2. Feature Engineering and Preprocessing

- Automatically handles missing data, scaling, and normalization.
- Includes techniques like feature encoding, one-hot encoding, and PCA (Principal Component Analysis).
- Allows users to configure preprocessing steps based on dataset type.

3. Model Training and Evaluation

- Users can select machine learning models (e.g., Random Forest, SVM, KNN, Neural Networks).
- Supports automatic model training using the uploaded dataset.
- Provides model evaluation with accuracy metrics, precision, recall, and F1-score.

4. Cross-validation and Auto-optimization

- Implements K-fold cross-validation to validate model performance.
- Automatically tunes model parameters using grid search or random search for optimal performance.

5. Model Testing and Prediction

- Allows users to test models on new data and generate predictions.
- Provides performance results such as confusion matrices, ROC curves, and feature importance.

6. Visualization and Results Reporting

- Includes interactive visualizations (e.g., graphs, charts) of model performance.
- Visualizes feature importance to highlight critical factors influencing predictions.
- Provides detailed reports of model performance for documentation and sharing.

7. Model Export and Integration

- Enables users to save and load trained models for future use.
- Supports exporting models in formats like pickle or ONNX.

- Allows integration with other systems for real-time predictions.

8. User Interface (UI) and Dashboard

- Provides a clean, intuitive web interface for users to interact with the platform.
- Includes a personalized dashboard for each user, displaying their datasets, models, and results.
- Offers configuration options for setting model parameters, dataset specifications, and preprocessing options.

9. Account Management and User Authentication

- Provides user account creation, login, and password management.
- Implements role-based access control (RBAC) for varying user permissions (e.g., admin, researcher).
- Supports multi-factor authentication (MFA) for enhanced security.

10. Collaboration Features

- Allows users to share datasets, models, and results with other researchers.
- Includes team collaboration features, such as access control and real-time updates.

Product Decomposition

The **Product Decomposition** breaks down the product features into smaller, actionable components:

Feature	Component	Description
Dataset Upload and Management	- File Upload Interface - Data Validation & Preprocessing	Provides an interface to upload files and ensures data quality before use.
Feature Engineering and Preprocessing	- Missing Data Handling - Feature Encoding & Normalization	Automatically processes and prepares data for model training.
Model Training and Evaluation	- Model Selection Interface - Model Training Engine	Allows users to select models and trains them based on uploaded datasets.
Cross-validation and Auto-optimization	- Cross-validation Engine - Parameter Optimization Engine	Validates models using K-fold cross-validation and optimizes parameters.
Model Testing and Prediction	- Testing Interface - Prediction Engine	Allows users to test models and generate predictions on new data.
Visualization and Results Reporting	- Graphical Plotting - Reporting Engine	Provides graphical representation of results and performance reports.
Model Export and Integration	- Model Exporter - Integration Interface	Exports trained models in various formats and allows integration with other systems.

UI and Dashboard	- Web Interface - Personalized Dashboard	The main interface for user interactions, displaying datasets, models, and results.
Account Management and User Authentication	- User Registration & Login - Role-based Access Control	Manages user accounts, roles, and permissions, with secure login options.
Collaboration Features	- Sharing Interface - Team Collaboration Tools	Allows sharing of resources and facilitates team-based research.

Table 9

This breakdown allows your development team to focus on each feature and its sub-components, making it easier to implement, test, and refine the system.

Chapter 2:

Software Requirement Specification (For Object Oriented Approach)

2.1 Introduction:

Genetic-Insight is a Software-as-a-Service (SaaS) platform designed to provide an easy-to-use, web-based solution for classifying proteins and genes through machine learning models. The platform enables researchers and the scientific community to upload datasets, configure module settings via a graphical interface, and perform various tasks such as model training, testing, evaluation, and prediction, all without needing to write any code.

The project is developed using an object-oriented approach to ensure a modular, reusable, and scalable architecture. This allows for easier maintenance, future updates, and extension of functionalities, supporting a variety of user needs in the field of bioinformatics and genomics.

The main objective of **Genetic-Insight** is to provide a comprehensive, user-friendly solution for automating the process of genomic data analysis and protein/ gene classification. This platform leverages state-of-the-art machine learning algorithms, such as Random Forest, Support Vector Machines (SVM), Neural Networks, and K-Nearest Neighbors (KNN), while offering features like dataset splitting, feature selection, cross-validation techniques, and model evaluation. The system also integrates result visualization tools, providing users with insightful visual outputs such as graphs and statistical plots.

This document outlines the software requirements for the **Genetic-Insight** platform using an object-oriented methodology. The following sections of the Software Requirement Specification (SRS) provide detailed information about the system's objectives, its features, external interfaces, performance attributes, and any assumptions and dependencies.

2.1.1 Systems Specifications

The following are the clauses that must be included while describing the system specifications.

Introduction

Genetic-Insight is an innovative Software-as-a-Service (SaaS) platform designed to provide machine learning-based solutions for the classification of proteins and genes. It caters primarily to researchers, scientists, and bioinformaticians working in the fields of genomics and bioinformatics. The platform offers an intuitive, web-based interface, enabling users to upload genomic datasets, train machine learning models, and evaluate their performance without needing to write any code.

The system incorporates cutting-edge machine learning algorithms such as Random Forest, Support Vector Machines (SVM), Neural Networks, and K-Nearest Neighbors (KNN) for protein and gene classification. Its aim is to make genomic data analysis accessible, scalable, and automated, helping users gain valuable insights from large datasets with minimal effort.

Genetic-Insight is developed with a modular, object-oriented approach to ensure scalability, reusability, and easy maintenance. This architecture supports future expansions in terms of functionality, such as integrating new machine learning algorithms or supporting additional data formats.

Organizational Chart

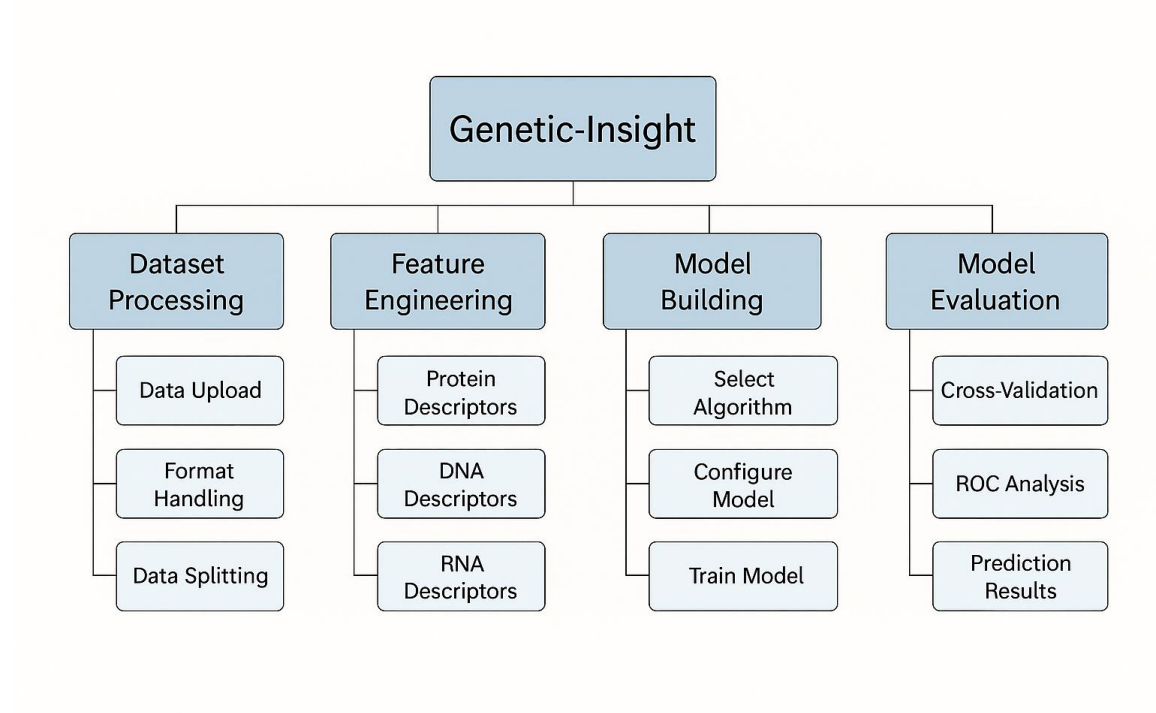


Figure 3

2.1.2. Identifying External Entities

The identification of external entities is an essential step in understanding the interactions between the system and its environment. External entities are any components, systems, or users that interact with **Genetic-Insight**, either by providing data to the system or receiving data from it. Identifying these entities ensures that the system's design accounts for all necessary interactions and interfaces.

The identification of external entities will be done in two phases, as outlined below:

a. Over Specify Entities from Abstract:

In the first phase, we identify all possible external entities based on the information provided in the **Abstract**. These entities are derived from the system's major components and the business needs described earlier.

Based on the abstract of **Genetic-Insight**, the following external entities can be over-specified:

1. **Users (Researchers and Scientists):**
 - Researchers who will interact with the platform, uploading datasets, configuring models, and analyzing results.
 - Scientists who may use the system to classify genomic data and extract insights for their research.
2. **Genomic Data Sources:**
 - Various datasets in CSV or Excel formats containing genomic information, such as protein sequences, gene expressions, and DNA/RNA sequences, which users will upload for analysis.
3. **Machine Learning Algorithms (External Libraries/Modules):**
 - Pre-trained models or external machine learning libraries (such as Scikit-learn, TensorFlow, Keras) will be used to train and classify genomic data.
4. **External Visualization Tools:**
 - External libraries for rendering visualizations such as graphs and plots (e.g., Matplotlib, Plotly, Seaborn).
5. **System Administrators:**
 - Admin users who will oversee the management of the platform, user permissions, and monitor system health.
6. **Third-Party APIs:**
 - APIs or services that may be integrated into the platform for additional functionality, such as cloud storage for dataset handling, authentication services, or external data retrieval.

b. Perform Refinement:

In the second phase, after over-specifying the entities, we refine them based on the specific business logic of the system. This involves analyzing each entity's role and relevance to the core functionalities of the **Genetic-Insight** platform. The goal is to narrow down the list to the most essential and meaningful external entities, ensuring that they are aligned with the system's objectives.

Refined external entities for **Genetic-Insight** are:

1. **End Users (Researchers and Bioinformaticians):**
 - The core users of the system who will upload datasets, train machine learning models, evaluate the results, and interpret the data for their research purposes. This entity is central to the system, and the platform's design will be focused on meeting their needs for simplicity and efficiency.

2. Genomic Data Files:

- Users will interact with genomic data files (e.g., CSV, Excel) containing data on proteins, genes, and other genomic information. The system will focus on handling these datasets efficiently, ensuring data validation, preprocessing, and smooth integration into machine learning models.

3. Machine Learning Models:

- These are the core components of the system, such as Random Forest, SVM, KNN, and Neural Networks, which will be trained and tested on the genomic data provided by the users. This entity will be refined based on the algorithms selected for the system and the need for integration with external libraries and frameworks for model training.

4. Data Visualization Tools:

- The system will generate results visualizations, including graphs and plots, based on the outputs of machine learning models. This entity may involve external visualization libraries, but the focus will be on integrating these tools seamlessly within the platform to ensure a smooth user experience.

5. Authentication and Authorization Services:

- The platform will require user authentication (login/logout) and authorization (role-based access) to ensure secure interactions with the system. External services may be used for user authentication and profile management.

6. System Admins:

- Admins will have the responsibility for monitoring system usage, managing user permissions, updating system components, and troubleshooting. They are a vital part of the system's maintenance.

7. Cloud Services for Data Storage:

- The platform will require cloud storage solutions for storing user-uploaded datasets, model results, and user reports. External cloud services (e.g., AWS, Google Cloud) may be integrated into the system for scalable and secure data management.

This process of identifying and refining external entities ensures that the system will be designed to interact effectively with all necessary components. It also helps clarify the roles of various entities in achieving the system's goals.

2.1.3. Context Level Data Flow Diagram:

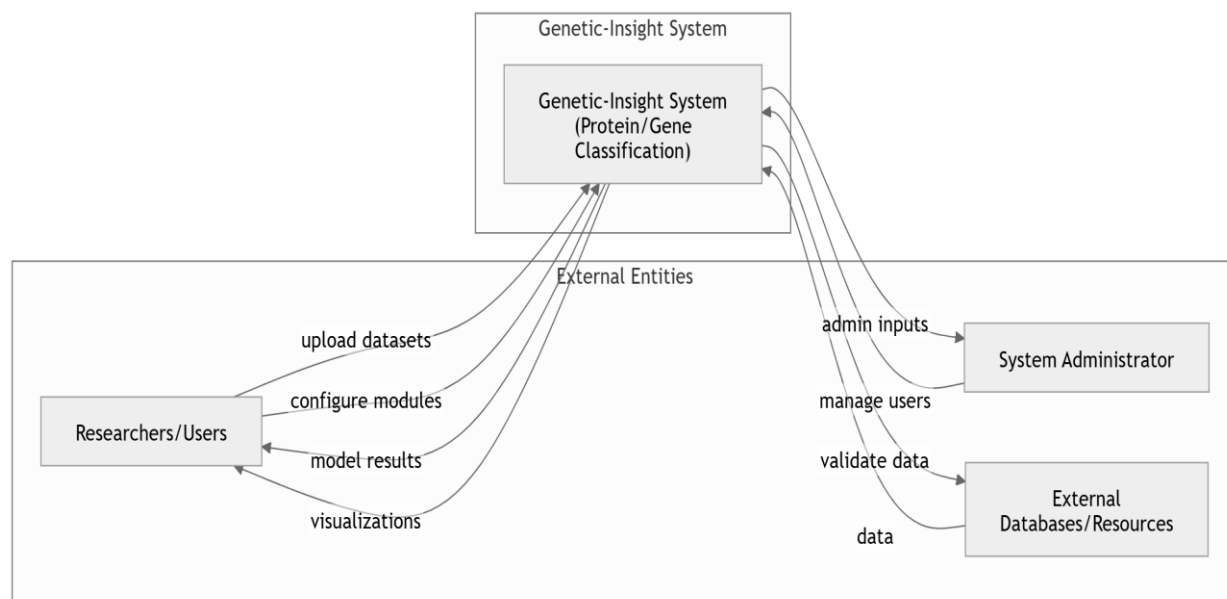


Table 10

2.2. Existing System

Currently, the process of genomic data analysis and protein/gene classification is labor-intensive and typically requires specialized expertise in bioinformatics and machine learning. Researchers and scientists must manually write code for dataset preprocessing, feature engineering, model selection, and evaluation, often using complex programming languages and tools like Python, R, or MATLAB.

There is also a gap in the market for an integrated, user-friendly platform that automates these tasks without requiring users to have extensive programming skills. Existing tools in the market may offer individual functionalities, but they often lack the seamless integration of dataset upload, model training, evaluation, and result visualization in a single platform.

Genetic-Insight aims to fill this gap by offering a comprehensive solution with an easy-to-use interface, automated model training and evaluation, and built-in visualization tools

2.2.1. Scope of the System

The scope of the Genetic-Insight system is divided into multiple phases to ensure structured and comprehensive development, focusing on the classification of proteins and genes through advanced machine learning techniques.

Phase I: Data Integration and Basic Analysis

Phase I focuses on establishing the foundational components of the system, enabling users to upload datasets and perform basic analysis for protein and gene classification.

- **Dataset Upload and Management:** Develop functionality for secure and efficient dataset upload by users, ensuring proper data handling and organization.
- **User Account Management:** Implement a secure authentication system for managing user accounts and ensuring data privacy.

- **Data Preprocessing Module:** Create a system for preprocessing datasets, including handling missing data, normalization, and feature extraction.
- **Basic Feature Analysis:** Develop basic modules to identify and extract relevant features from uploaded datasets for preliminary classification analysis.

Phase II: Advanced Machine Learning and Personalization

Phase II focuses on enhancing the system's analysis capabilities and introducing personalized recommendations for model selection and optimization.

- **Model Training and Evaluation:** Implement advanced machine learning algorithms (e.g., Random Forest, SVM, Neural Networks) for protein and gene classification, along with cross-validation and hyperparameter optimization.
- **Personalized Model Configuration:** Develop a system that allows users to configure training modules based on their datasets and specific requirements.
- **Feature Selection and Optimization:** Implement techniques to select the most relevant features and optimize model performance based on dataset characteristics.
- **Results Visualization:** Provide advanced data visualization tools to display model results, including accuracy, feature importance, and classification reports.
- **Progress Monitoring:** Build a tracking system to monitor improvements in classification accuracy over time as users refine their models.

Phase III: Comprehensive Solution and System Optimization

Phase III focuses on integrating all system components, optimizing performance, and improving usability.

- **Full System Integration:** Integrate all subsystems, including dataset management, machine learning modules, and user-facing features, into a cohesive web-based platform.
- **System Optimization:** Optimize the performance of the system, focusing on increasing algorithm accuracy, data processing efficiency, and overall platform responsiveness.
- **User Feedback and Collaboration Tools:** Develop features for users to provide feedback and collaborate with researchers for ongoing improvements to the system.
- **Real-Time Model Evaluation:** Implement real-time functionality for evaluating models and displaying immediate results for better decision-making.

2.2.2. Summary of Requirements:(Initial Requirements)

The proposed Genetic-Insight system must fulfill the following initial requirements:

1. User Account Management:

- The system must allow users to securely create, manage, and authenticate their accounts.
- User authentication should be robust, ensuring data privacy and security.

2. Dataset Upload and Management:

- Users must be able to upload datasets containing protein and gene data.
- The system must handle various data formats and ensure secure and efficient storage of uploaded datasets.

3. Data Preprocessing:

- The system must include a preprocessing module to clean and normalize datasets.
- Handling of missing data, outliers, and feature extraction must be automated.

4. Basic Feature Analysis:

- The system must include basic feature analysis modules to identify and extract key features from datasets for protein and gene classification.
- It should provide initial insights into the data before further analysis.

5. Model Training and Evaluation:

- The system must support the use of machine learning models (e.g., Random Forest, SVM, Neural Networks) for classifying proteins and genes.
- It should evaluate models using cross-validation techniques to ensure accurate results.

6. User-Friendly Interface:

- The system must provide a user-friendly interface that is intuitive and easy to navigate for researchers and users with limited technical knowledge.
- It must allow users to configure training modules based on their dataset and requirements.

7. Data Visualization:

- The system must include tools to visualize results, such as accuracy metrics, feature importance graphs, and classification reports.
- Users should be able to view the results of their models in clear and informative charts.

8. Real-Time Feedback:

- The system must offer real-time feedback on the performance of the models as users interact with the system and adjust their settings.

2.2.3. Identifying External Entities:

The identification of external entities for the **Genetic-Insight** system will be carried out in two phases, as explained below. This identification will be based on the abstract provided and will be refined through business logic considerations.

Phase 1: Over Specifying Entities from the Abstract

Based on the **Genetic-Insight** system's abstract and objectives, the following external entities can be identified:

- **User:** The person or researcher interacting with the system, uploading datasets, and analyzing genetic and protein data.
- **Dataset:** The data files containing information on proteins, genes, or other biological components uploaded by users.
- **Machine Learning Model:** The algorithms (e.g., Random Forest, SVM, Neural Networks) used for classifying proteins and genes.
- **Model Evaluation:** The process by which the system evaluates the performance of machine learning models, such as accuracy, precision, and recall.
- **System Interface:** The web-based interface that allows users to interact with the Genetic-Insight system, configure settings, and view results.
- **Results:** The output generated by the system after processing datasets, including

classification results, accuracy, and feature importance.

- **User Account:** The account through which users manage their profiles, settings, and data.
- **API (Application Programming Interface):** Any external system or tool that interacts with Genetic-Insight through an API for additional functionalities or integration.

Phase 2: Refining the Entities Based on Business Logic

After over-specifying the entities, we refine them according to the business logic of the **Genetic-Insight** system. The entities relevant to the system's core functions include:

- **User:** The entity that interacts with the system, uploads data, and configures machine learning models. The User is central to the platform's functionality.
- **Dataset:** The data input provided by users, which is processed by the system for analysis. It could include gene expression data, protein sequences, and other biological datasets.
- **Model:** The machine learning algorithms (e.g., Random Forest, SVM, Neural Networks) that are applied to the dataset to generate classifications and predictions.
- **Results:** The outcome of the analysis, including the model's performance metrics and insights derived from the data (e.g., feature importance, classification reports).
- **User Account:** Represents the user's profile and preferences, which are securely managed and linked to the datasets they upload and the analyses they perform.
- **System Interface:** The interface used by users to interact with the system, including visualizing results, configuring settings, and navigating the platform.

2.2.4. Capture "shall" Statements:

Para #	"Shall" Statement
1.0	The user shall register on the system to gain access.
1.0	The system shall allow users to log in using secure credentials.
1.0	The system shall provide an option to change the user password.
2.0	The user shall upload datasets (e.g., protein and gene data) to the system.
2.0	The system shall accept and securely store the uploaded datasets.
2.0	The system shall support multiple data formats for dataset upload.
3.0	The system shall preprocess the datasets by filtering noise, normalizing values, and handling missing data.
3.0	The system shall extract relevant features from the dataset for further analysis.
4.0	The system shall train machine learning models (e.g., Random Forest, SVM, Neural Networks) based on the uploaded dataset.
4.0	The system shall evaluate the models using cross-validation techniques to ensure accuracy.
4.0	The system shall display model performance metrics, including accuracy, precision, and recall.
5.0	The system shall provide a visualization of the model results, including charts and graphs showing feature importance.
5.0	The system shall allow users to view detailed reports of the classification results.
6.0	The user shall be able to manage their profile, including viewing and updating personal details.
6.0	The system shall securely store user account information and preferences.
6.0	The system shall allow users to track their progress over time based on their dataset analysis and model evaluations.
7.0	The system shall provide real-time feedback on the progress of dataset processing and model training.
7.0	The system shall notify the user upon the completion of dataset analysis or model evaluation.
8.0	The system shall ensure the security and privacy of user data and datasets.
8.0	The system shall comply with relevant data protection regulations.

2.2.5. Allocate and Priorities Requirements:

Requirement Description	"Shall" Statement	Use Case	Priority
Dataset Upload and Integration	The system shall allow users to upload datasets (e.g., protein and gene data) to the system.	Data Integration	Highest
Machine Learning Model Training	The system shall train machine learning models (e.g., Random Forest, SVM, Neural Networks) based on the uploaded dataset.	Model Training	Highest
Cross-Validation and Evaluation	The system shall evaluate the models using cross-validation techniques to ensure accuracy.	Model Evaluation	Highest
Feature Extraction and Preprocessing	The system shall preprocess datasets by filtering noise, normalizing values, and handling missing data.	Data Preprocessing	High
Result Visualization	The system shall provide a visualization of the model results, including charts and graphs showing feature importance.	Result Visualization	High
User Profile and Data Management	The system shall securely store and manage user account information and datasets.	User Account Management	Medium
Historical Data Analysis	The system shall record and analyze historical user data to improve recommendations and insights.	Historical Data Analysis	Medium
Progress Tracking	The system shall allow users to track their progress over time based on their dataset analysis and model evaluations.	Progress Tracking	Medium
Report Generation and Export	The system shall support the generation of detailed reports summarizing the model's performance and user data.	Reporting and Export	Medium
Security and Data Privacy	The system shall ensure secure storage and transmission of sensitive user data using encryption mechanisms.	Security Management	Low
User-Friendly Interface	The system shall provide a user-friendly interface for users to interact with the platform and view results.	User Interface Development	Low

2.2.6. Requirements Traceability Matrix:

Requirement ID	Requirement Description	Design Element	Implementation Component	Testing Procedure
FR-001	Allow users to upload datasets (e.g., protein and gene data) to the system.	Data Integration Module	Dataset Upload and Integration	Test dataset upload functionality and error handling
FR-002	Train machine learning models (e.g., Random Forest, SVM, Neural Networks) based on the uploaded dataset.	Model Training Module	ML Model Implementation	Validate model training, accuracy, and predictions
FR-003	Evaluate the models using cross-validation techniques to ensure accuracy.	Model Evaluation Module	Cross-Validation Implementation	Test model evaluation and cross-validation results
FR-004	Preprocess datasets by filtering noise, normalizing values, and handling missing data.	Data Preprocessing Module	Data Cleaning and Normalization Code	Verify data preprocessing accuracy
FR-005	Provide visualizations of model results, including charts and graphs showing feature importance.	Result Visualization Module	Visualization Tools (Graphs, Charts, Dashboards)	Validate visualization accuracy and interactivity
FR-006	Securely store and manage user account information and datasets.	User Account Management Module	User Authentication and Data Management Code	Test account creation, login, and data storage
FR-007	Record and analyze historical user data to improve recommendations and insights.	Historical Data Analysis Module	Data Tracking and Historical Analysis Code	Verify historical data recording and analysis

FR-008	Track user progress over time based on dataset analysis and model evaluations.	Progress Tracking Module	User Progress Tracking and Reporting Code	Test progress tracking functionality and accuracy
FR-009	Generate detailed reports summarizing the model's performance and user data.	Reporting and Export Module	Report Generation and Export Code	Validate report accuracy and export functionality
NFR-001	Ensure secure storage and transmission of sensitive user data using encryption mechanisms.	Security Module	Data Encryption and Authentication Code	Perform penetration testing and data privacy testing
NFR-002	Ensure system reliability and scalability, especially for large datasets.	Backend Architecture	Cloud-Based Scalable Infrastructure	Conduct load testing and reliability checks
NFR-003	Maintain consistent performance with minimal delays in data processing and model evaluation.	System Optimization Procedures	Performance Tuning Code	Perform stress and performance testing

2.2.7. High Level Use-case Diagram:

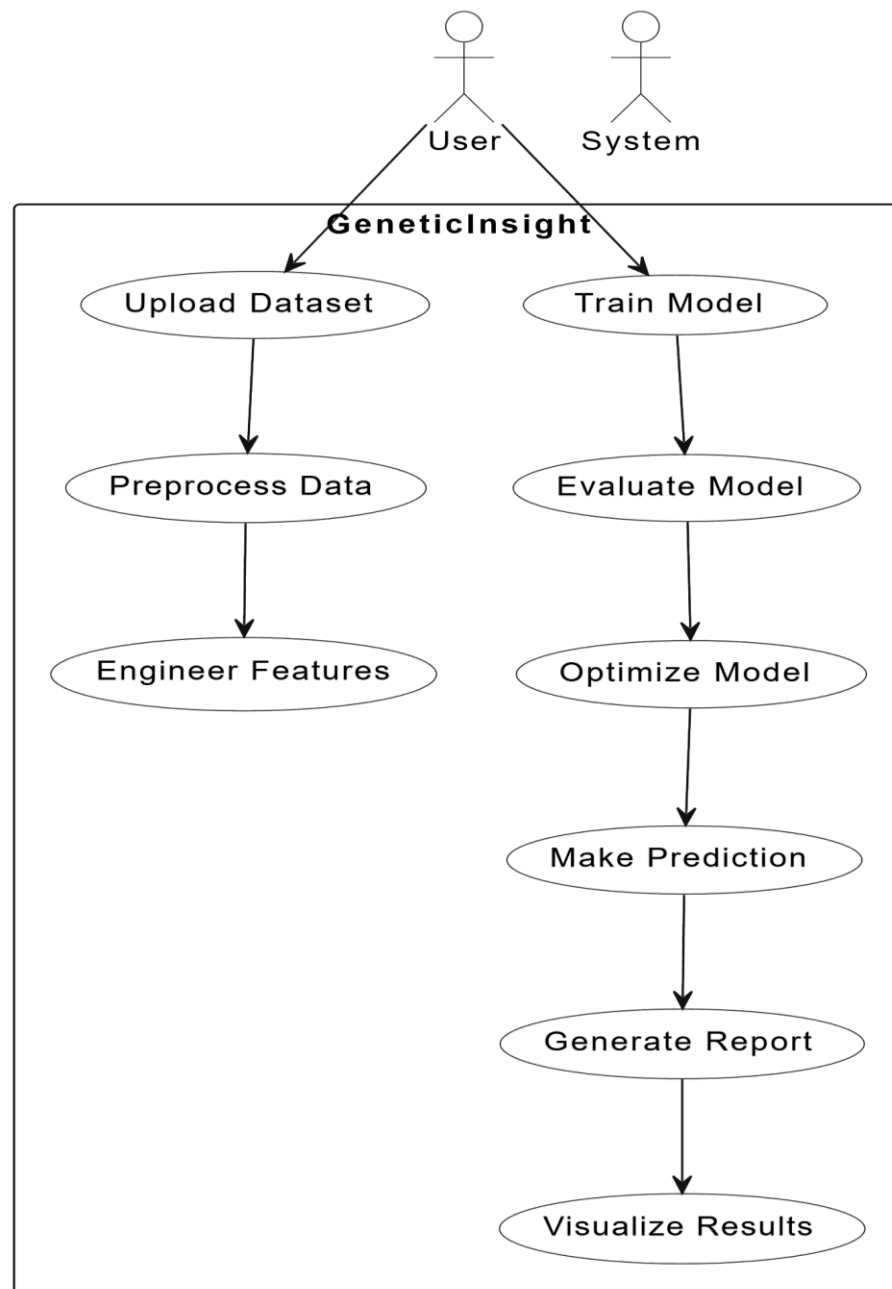


Figure 4

2.2.8. Use-case Description

UC01: Upload Datasets

Use Case ID	UC01
Use Case Name	Upload Datasets
Brief Description	Allows users to upload their protein, gene, or RNA datasets for processing and model training.
Preconditions	- The user must be logged into the system. - The system must support the file types (e.g., CSV, Excel, etc.) for dataset uploads.
Basic Flow	1. The user navigates to the dataset upload page. 2. The user selects the dataset file from their local system. 3. The user clicks the "Upload" button. 4. The system processes the file and confirms successful upload.
Alternate Flows	- Invalid file format: The system prompts the user to upload a valid file type. - Upload error: The system notifies the user and prompts them to retry.
Post Conditions	- The dataset is successfully uploaded and stored in the system. - The user can proceed to configure model settings.

UC02: Configure Settings

Use Case ID	UC02
Use Case Name	Configure Settings
Brief Description	Users can configure settings for feature engineering, preprocessing, and other model training parameters.
Preconditions	- The user must be logged into the system. - The dataset must be uploaded successfully.
Basic Flow	1. The user navigates to the configuration page. 2. The user selects feature engineering options and preprocessing methods. 3. The user reviews and confirms the settings. 4. The system stores the selected configurations for model training.
Alternate Flows	- Invalid settings: The system notifies the user to check their selections.
Post Conditions	- The configuration settings are saved and ready for model training.

UC03: Train Model

Use Case ID	UC03
Use Case Name	Train Model
Brief Description	Allows users to train a model using the uploaded dataset and selected configurations.
Preconditions	- The user must be logged into the system. - A valid dataset must be uploaded. - The user must have configured the settings for model training.
Basic Flow	1. The user navigates to the model training page. 2. The user selects the model type (e.g., Random Forest, SVM, etc.). 3. The user clicks the "Train" button to start the training process. 4. The system begins model training based on the dataset and configurations.
Alternate Flows	- Training error: The system notifies the user and suggests retrying the process. - Timeout: If the training takes too long, the system asks the user to wait or retry later.
Post Conditions	- The model is trained and ready for evaluation. - The user can view the training progress or results.

UC04: Evaluate Model

Use Case ID	UC04
Use Case Name	Evaluate Model
Brief Description	Allows users to evaluate the performance of a trained model using metrics like accuracy, cross-validation, etc.
Preconditions	- The user must be logged into the system. - The model must be successfully trained.
Basic Flow	1. The user navigates to the evaluation page. 2. The system presents evaluation metrics (accuracy, F1 score, etc.). 3. The user reviews the evaluation results.
Alternate Flows	- Evaluation failure: The system notifies the user and suggests retrying or adjusting the model.
Post Conditions	- The evaluation results are available for the user to review. - The user can decide whether to proceed with the model or retrain it.

UC05: Visualize Result

Use Case ID	UC05
Use Case Name	Visualize Results
Brief Description	Users can view results through graphs, charts, and other visual tools for better understanding.
Preconditions	- The user must be logged into the system. - The model must be trained and evaluated.
Basic Flow	1. The user navigates to the results visualization page. 2. The system generates visual representations of the evaluation metrics and results. 3. The user reviews the visual results.
Alternate Flows	- No data available: If no results are available, the system notifies the user.
Post Conditions	- The user can visually interpret the model's performance and insights.

UC06: Export Model

Use Case ID	UC06
Use Case Name	Export Model
Brief Description	Allows users to save and export their trained model for later use or deployment.
Preconditions	- The user must be logged into the system. - The model must be successfully trained.
Basic Flow	1. The user navigates to the model export page. 2. The user selects the model export format (e.g., .pkl, .h5, etc.). 3. The user clicks the "Export" button. 4. The system exports the model and provides a download link.
Alternate Flows	- Export error: The system notifies the user of any export failures.
Post Conditions	- The model is successfully exported and available for use or deployment.

UC07: View Documentation

Use Case ID	UC07
Use Case Name	View Documentation
Brief Description	Provides users access to the documentation, guides, and help resources related to the platform.
Preconditions	- The user must be logged into the system.
Basic Flow	1. The user navigates to the documentation section. 2. The system displays available documentation, tutorials, and FAQs. 3. The user browses or searches for relevant information.
Alternate Flows	- No documentation available: The system notifies the user that documentation is not available.
Post Conditions	- The user is able to access and view helpful resources.

Chapter 3:

Design Document (For Object Oriented Approach)

3.1. Introduction:

The third deliverable focuses on the software design, which takes the insights gained from the system analysis phase and converts them into a concrete plan for the solution. In the previous deliverable, we analyzed the problem domain and understood its current situation. With that foundation, we can now move forward with an object-oriented approach to design a robust solution. This section will cover several important artifacts that form the basis of the object-oriented design. These artifacts ensure that we have a clear, effective, and reusable design before implementation begins.

The following artifacts will be covered in detail:

1. Domain Model
2. System Sequence Diagram
3. Sequence Diagram
4. Collaboration Diagram
5. Operation Contracts
6. Design Class Diagram
7. State Transition Diagram
8. Data Model

Now we discuss these artifacts one by one as follows:

1. Domain Model

- The domain model serves as a conceptual model of the problem domain. It identifies the main objects, entities, and relationships in the system. The domain model provides a high-level view of the system's components without diving into technical details. It focuses on the objects and their interactions that define the domain we're addressing.

2. System Sequence Diagram

- The system sequence diagram illustrates the flow of messages between actors and the system in response to an event. It shows the interaction between the system and external actors, outlining how inputs are received and processed in the system. This diagram provides an overview of the system's dynamic behavior in response to user actions.

3. Sequence Diagram

- A sequence diagram is used to represent the interactions between objects in a specific use case or scenario. It shows the objects involved in the interaction, the messages exchanged between them, and the order of these messages. Sequence diagrams help clarify how the system components collaborate to achieve a specific functionality.

4. Collaboration Diagram

- A collaboration diagram, also known as a communication diagram, emphasizes the relationships between objects rather than the sequence of messages. It focuses on the structure and the flow of messages between objects to achieve a particular goal or complete a process. This diagram is useful for understanding the collaboration between objects in a system.

5. Operation Contracts

- Operation contracts define the preconditions, postconditions, and invariants of each operation in the system. They clarify the behavior of methods and their expectations, including what must be true before and after the method is executed. These contracts ensure that all operations are well-defined and maintain consistency throughout the system.

6. Design Class Diagram

- The design class diagram provides a detailed view of the system's classes, their attributes, methods, and relationships. It includes more specific information compared to the domain model, such as visibility modifiers (public, private), data types, and method signatures. This diagram is essential for creating the actual software components during the implementation phase.

7. State Transition Diagram

- A state transition diagram models the states that an object can be in and the transitions between those states based on events. It helps understand how objects behave in response to various inputs and how they transition through different states during their lifecycle. This diagram is particularly useful for systems with complex state-based behavior.

8. Data Model

- The data model defines the structure of the data used by the system, including entities, relationships, and data flow. It serves as a blueprint for how data will be stored, retrieved, and manipulated within the system. The data model typically includes concepts like tables, columns, and foreign key relationships for relational databases or other storage mechanisms.

3.2. Domain Model

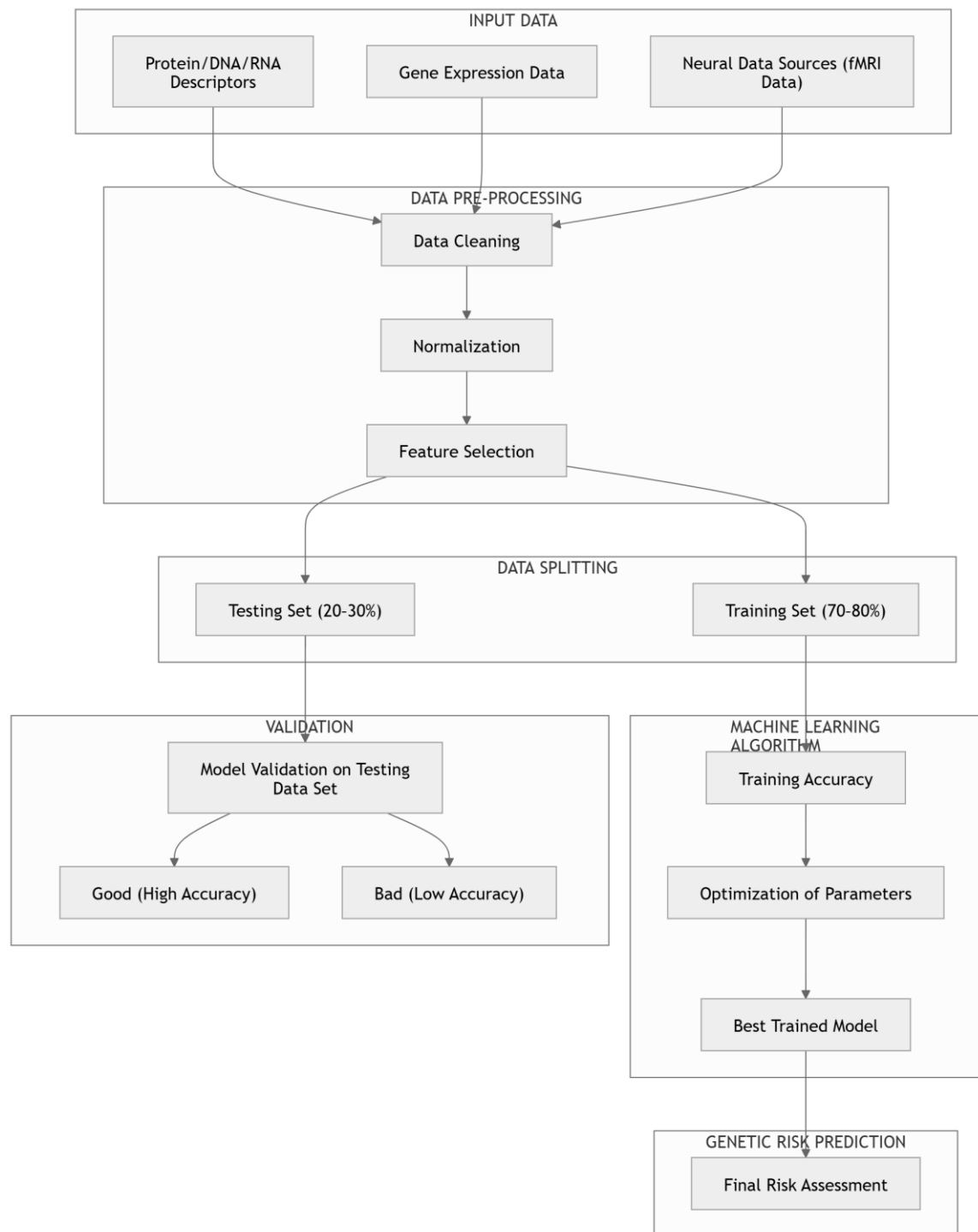


Figure 5

3.3. System Sequence Diagram

Key Elements in SSD:

1. **Actors:**
 - Represent external entities that interact with the system (e.g., users, automated systems, robots).
 - Actors are shown on the left side of the diagram with their names and roles.
 - Automated actors or systems may have a horizontal line through their head to distinguish them.
2. **System:**
 - Represented on the right side of the diagram. It is the entity responding to inputs.
 - The system handles the sequence of operations triggered by the actors.
3. **Events (Messages):**
 - Arrows between the actor and the system represent messages or events exchanged during the interaction.
 - Events are listed in the order in which they occur, starting from the top.
 - The sequence of events helps trace the flow of actions triggered by user inputs.
4. **Time Sequence:**
 - The SSD is structured in a vertical timeline, where events occur from top to bottom.
 - The first event occurs at the top, and subsequent events follow in a sequential manner.
5. **System Events:**
 - These are key actions or operations that trigger a response from the system.
 - They represent significant events that change the state of the system.

Purpose of SSD:

- It provides a clear understanding of how the system responds to various inputs over time.
- Helps in visualizing the order of operations and understanding the flow of a specific use case.
- It also defines the system's behavior in response to external triggers.

Use Case Integration:

- If included, use case text can be placed on the left side of the diagram, aligning with the events represented.
- This helps link specific use cases to the sequence of events in the diagram.

3.4. Sequence Diagram

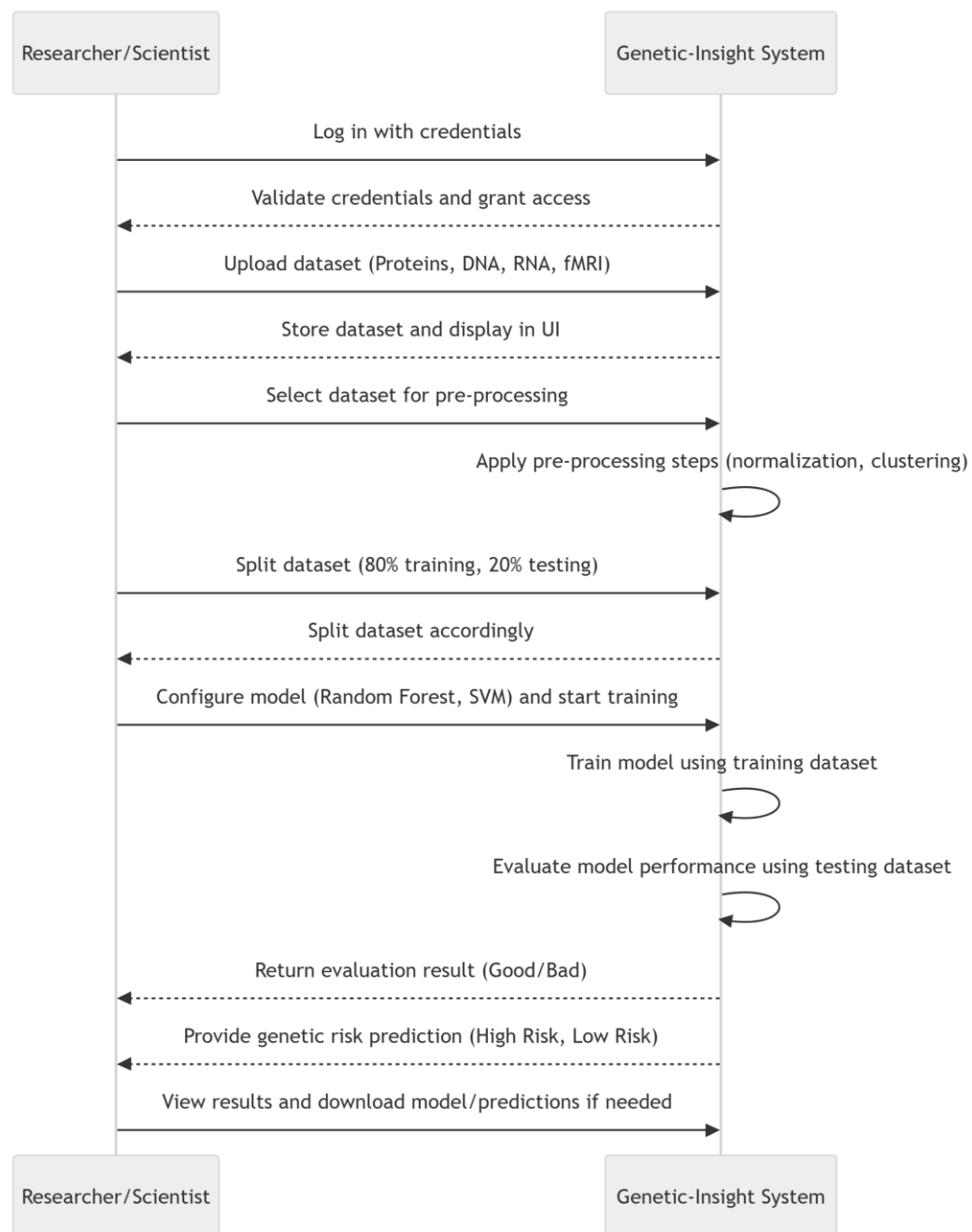


Figure 6

3.4.1. Defining a Sequence diagram

Step 1: Identify Actors and Objects

Actors:

1. **User:** The researcher or scientist interacting with the Genetic-Insight platform (e.g., uploading datasets, configuring models, and analyzing results).

Objects:

1. **Genetic-Insight Platform:** The core system that processes the data, trains models, and provides results.
2. **Dataset:** The class representing the genetic data (Proteins, DNA, RNA) uploaded by the user.
3. **Pre-Processing Module:** The component responsible for data normalization, clustering, and feature encoding.
4. **Model Trainer:** The component responsible for training machine learning models based on the user's configuration.
5. **AI Model:** The machine learning model (e.g., Random Forest, SVM, Neural Networks) that analyzes the data for genetic risk prediction.
6. **Result:** The final output generated by the AI model, such as a risk score or classification.
7. **Database:** The storage where the dataset, model parameters, and results are stored.

Step 2: Define Sequence of Actions

Here's how the sequence of actions for Genetic-Insight could unfold:

1. **User uploads dataset:** The user uploads a genetic dataset (e.g., Proteins, DNA, RNA) to the Genetic-Insight platform.
2. **Dataset is passed to the Pre-Processing Module:** The uploaded dataset is passed to the pre-processing module for normalization, clustering, and feature encoding.
3. **Pre-Processing Module sends data to the Model Trainer:** The processed data is then sent to the model trainer.
4. **Model Trainer configures and trains the AI Model:** The Model Trainer configures the selected machine learning model and trains it on the processed data.
5. **AI Model analyzes the data:** The trained AI Model is used to analyze the data and generate predictions (e.g., genetic risk score).
6. **Result is returned to the Platform:** The prediction result is sent back to the Genetic-Insight platform.
7. **Result displayed or stored:** The result is either displayed to the user or stored in the database for future reference.

Step 3: Draw the Sequence Diagram

The sequence diagram for Genetic-Insight will follow this flow:

1. **User** uploads the dataset to the **Genetic-Insight Platform**.
2. **Platform** communicates with the **Dataset** object to retrieve the data.
3. **Platform** passes the dataset to the **Pre-Processing Module** for normalization and feature encoding.
4. The **Pre-Processing Module** processes the data and sends it to the **Model Trainer**.

5. The **Model Trainer** configures the AI Model and trains it using the processed data.
6. The **AI Model** analyzes the data and sends the result back to the **Genetic-Insight Platform**.
7. The **Platform** either displays the result to the **User** or stores the result in the **Database** for later use.

Key Points for the Sequence Diagram:

1. **Messages:**
 - The flow of data from the **User** to the **Platform**, from the **Platform** to the **Pre-Processing Module**, from the **Pre-Processing Module** to the **Model Trainer**, and from the **Model Trainer** to the **AI Model**.
 - The result generated by the **AI Model** is either displayed to the **User** or stored in the **Database**.
2. **Activation Boxes:**
 - The **Genetic-Insight Platform**, **Pre-Processing Module**, **Model Trainer**, and **AI Model** will have activation boxes showing when they are processing data.
3. **Lifelines:**
 - The vertical dashed lines represent the lifelines of the **User**, **Genetic-Insight Platform**, **Dataset**, **Pre-Processing Module**, **Model Trainer**, **AI Model**, **Result**, and **Database**.
4. **Loops:**
 - If there are repetitive processes, such as multiple rounds of model training or data pre-processing, you can indicate them using loops. However, for simplicity, we'll stick with a linear flow for this use case.

Final Diagram Flow:

1. The **User** uploads or selects the dataset via the **Genetic-Insight Platform**.
2. The **Platform** communicates with the **Dataset** to retrieve the data.
3. The **Platform** passes the dataset to the **Pre-Processing Module** for processing.
4. The **Pre-Processing Module** sends the processed data to the **Model Trainer**.
5. The **Model Trainer** configures and trains the **AI Model** with the processed data.
6. The **AI Model** analyzes the data and returns the result to the **Genetic-Insight Platform**.
7. The **Platform** displays the result to the **User** or stores it in the **Database**.

3.5. Collaboration Diagram

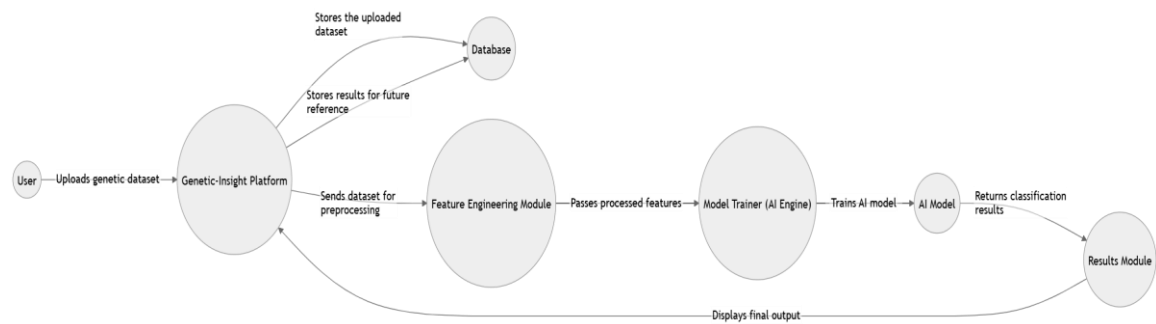


Figure 7

3.5.1. Contents of Collaboration Diagrams

A **Collaboration Diagram** (also known as a **Communication Diagram**) represents the interactions between objects or components in a system. It emphasizes the relationships and the flow of messages between them.

Key Elements of a Collaboration Diagram:

1. **Objects/Components (Nodes):**
 - Represented as **circles or rectangles**.
 - Each node represents an entity in the system (e.g., **User, Database, AI Model**).
2. **Links (Connections):**
 - **Lines** between objects indicate relationships or data exchange.
 - Arrows show the **direction** of communication.
3. **Messages (Interactions):**
 - Labeled arrows **describe data flow and actions** between components.
 - Numbering may be used to show **sequence of interactions**.
4. **Actors (Users or External Entities):**
 - Represented as a **separate node**, often connected to the main system.
 - Interacts with the system by **sending or receiving data**.
5. **Design Considerations:**
 - Use **consistent colors** (e.g., **blue circles** for components, **black lines** for connections).
 - Maintain **clear labeling** and **proper spacing** for readability.
 - Ensure a **logical flow** of interactions.

A collaboration diagram that describes part of the flow of events of the use case Receive Deposit Item in the Recycling-Machine System.

3.6. Operation Contracts

Operation 1: Upload Genetic Dataset

Attribute	Details
Name	Upload Genetic Dataset
Responsibilities	Allows the user to upload a genetic dataset for processing.
Cross References	Use Case: "Upload Dataset"
Exceptions	Invalid file format, file size exceeds limit
Preconditions	The user is logged into the system.
Postconditions	The dataset is stored in the database, and a reference ID is generated.

Table 11

Operation 2: Process Dataset for Feature Engineering

Attribute	Details
Name	Process Dataset
Responsibilities	Performs feature extraction, normalization, and selection.
Cross References	Use Case: "Feature Engineering"
Exceptions	Corrupt dataset, missing values
Preconditions	The dataset is uploaded and stored in the database.
Postconditions	Processed features are generated and stored for model training.

Table 12

Operation 3: Train AI Model

Attribute	Details
Name	Train Model
Responsibilities	Trains a machine learning model using the processed dataset.
Cross References	Use Case: "Train Model"
Exceptions	Insufficient data, model convergence failure
Preconditions	The dataset has been preprocessed.
Postconditions	The trained model is stored in the system for classification.

Table 13

Operation 4: Classify Genetic Data

Attribute	Details
Name	Classify Data
Responsibilities	Uses the trained AI model to classify genetic sequences.
Cross References	Use Case: "Classify Genetic Sequences"
Exceptions	Model not trained, input data format error
Preconditions	A trained model exists.
Postconditions	Classification results are generated and stored in the results module.

Table 14

Operation 5: Display Results to User

Attribute	Details
Name	Display Results
Responsibilities	Shows classification results and visualizations to the user.
Cross References	Use Case: "View Results"
Exceptions	No results found, database error
Preconditions	The classification process is completed.
Postconditions	Results are displayed on the UI, and users can download reports.

Table 15

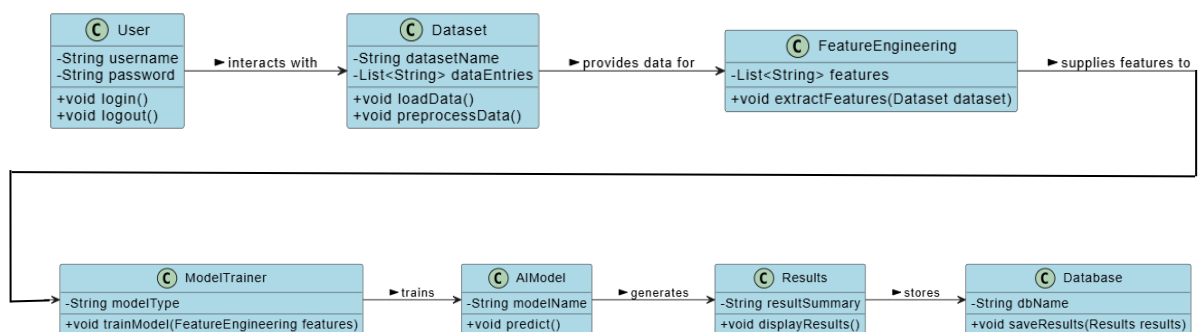
3.7. Design Class Diagram**Classes and Their Responsibilities**

Figure 8

Class Name	Attributes	Methods	Description
User	userID, name, email, role	uploadDataset(), viewResults()	Represents researchers using the system.
Dataset	datasetID, name, format, size	preprocess(), extractFeatures(), normalize()	Stores and processes genetic data.
FeatureEngineering	features[], selectionMethod	selectFeatures(), reduceDimensionality()	Performs feature selection and dimensionality reduction.
ModelTrainer	modelID, modelType, status	trainModel(), validateModel(), saveModel()	Trains ML models based on input data.
AIModel	modelID, accuracy, parameters	predict(), loadModel()	Performs classification on genetic data.
Results	resultID, classification, report	generateReport(), visualizeResults()	Stores and presents classification outputs.
Database	storageType, recordsCount	saveData(), retrieveData(), updateData()	Stores datasets, models, and results.

Table 16

Key Features in the Diagram:

1. Base Class and Inheritance

- **Abstract Class: Genetic Data Processor**
 - **Purpose:** Acts as a base class providing common methods like process Data () and extract Features (), which specialized components inherit.
- **Inheritance:**
 - **Feature Engineering Module** extends **Genetic Data Processor**
 - This represents a specialized version of Genetic Data Processor that performs preprocessing and feature extraction on genetic datasets.

2. Composition

- **Dataset**
 - Represents the uploaded genetic data, containing attributes like sequence Type (e.g., DNA, RNA, Protein) and data Points.
- **Genetic Model**
 - Has a **composition** relationship with Dataset:
 - (filled diamond) indicates Genetic Model owns Dataset, meaning the dataset's lifecycle is tied to the model.
 - Methods like train Model () and evaluate Model () help train and test AI models using genetic data.

3. Aggregation

- **User**
 - Represents user information, such as name, affiliation, and role, along with get Profile () to fetch user details.
- **Genetic Insight Platform**
 - Has an **aggregation** relationship with User:
 - ◦ (open diamond) denotes that multiple users can interact with the platform, but their existence is independent.

4. Association

- **Results Module**
 - Contains methods like generate Report () and download Results () to provide insights to researchers.
- **Relationship:** AIModel --> Results Module
 - The arrow denotes an association where the trained AI model generates results, but Results Module exists independently.

5. Dependency

- **Data Visualizer**
 - Provides visualization tools through plot Results (data: Dataset) to display classification insights.
- **Relationship:** Genetic Model.> Data Visualizer
 - A dashed arrow represents a dependency, meaning Genetic Model temporarily relies on Data Visualizer for output visualization.

6. Interfaces

- **Authentication**
 - Represents an interface with methods: login () and logout ().
- **User Manager implements Authentication**
 - Adds functionalities like register (), ensuring secure user authentication.
- **Relationship:** Genetic Insight Platform --> Authentication
 - The platform depends on Authentication for user login/logout management.

3.8. State chart diagram

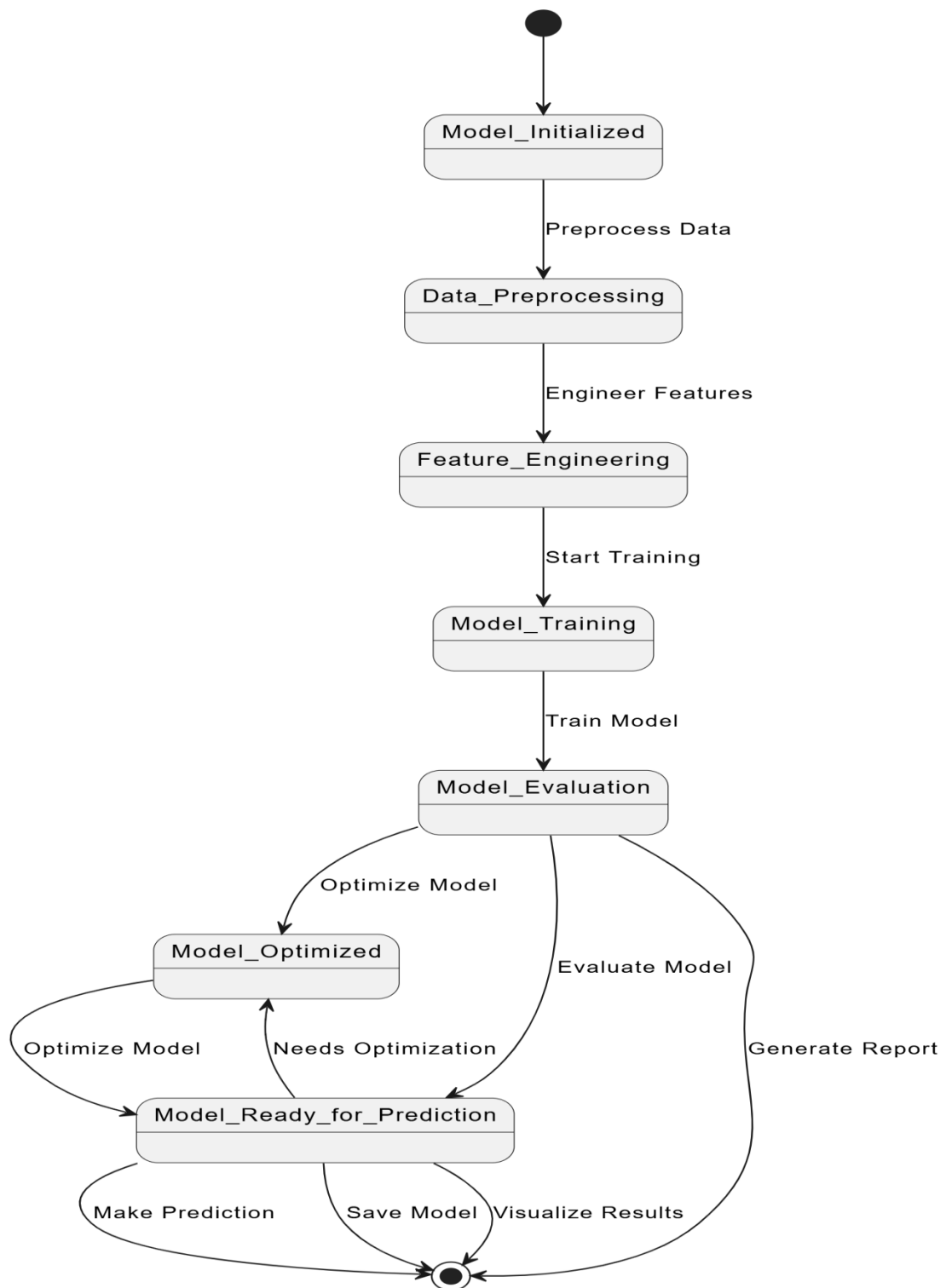


Figure 9

Explanation of State Chart Diagram

The **State Chart Diagram** for the **Genetic-Insight** project illustrates the dynamic behavior of a machine learning model as it moves through various stages during the process of training, evaluation, and prediction. The system revolves around processing biological datasets for protein and gene classification using machine learning models. The diagram defines the lifecycle of a model and the key events triggering state transitions.

States and Transitions:

1. **Model Initialized:**
 - This is the initial state when a user uploads a dataset and initializes the model.
 - The event '**Start Training**' triggers the transition to the next state, indicating that the model is ready to begin processing.
2. **Data Preprocessing:**
 - In this state, the data undergoes necessary preprocessing steps, such as cleaning, normalization, and splitting.
 - The event '**Preprocess Data**' leads to this state, and once the data is ready, the system transitions to **Feature Engineering**.
3. **Feature Engineering:**
 - During this stage, relevant features are extracted and transformed from the raw data, such as protein descriptors or gene sequences, which are critical for accurate predictions.
 - Once feature engineering is complete, the model moves to the **Model Training** state.
4. **Model Training:**
 - The model is trained using the prepared features and selected algorithms (e.g., Random Forest, SVM, Neural Networks).
 - The event '**Train Model**' triggers this state, and upon successful completion of the training, the model transitions to **Model Evaluation**.
5. **Model Evaluation:**
 - In this state, the trained model is evaluated against a test dataset using various metrics like accuracy, precision, recall, etc.
 - The '**Evaluate Model**' event causes the model to enter this state, and once evaluation is done, the model may move to **Model Optimized** or back to **Model Training** if improvements are required.
6. **Model Optimized:**
 - If the evaluation results show that the model can be improved, this state is triggered by the '**Optimize Model**' event.
 - In this state, hyperparameter tuning and other optimization techniques (e.g., cross-validation, grid search) are applied to enhance the model's performance.
 - After optimization, the model transitions to **Model Ready for Prediction**.
7. **Model Ready for Prediction:**
 - This is the final state where the model is fully trained, optimized, and ready to make predictions on new, unseen data.
 - The event '**Make Prediction**' marks the completion of the process, and at this point, the model can be used to classify proteins or genes.

Actions and Outputs:

- **Save Model:** After the model is trained, it can be saved for future use or deployment.
- **Generate Report:** Reports detailing the performance metrics and feature importance can be generated at the **Model Evaluation** and **Model Optimized** stages.
- **Visualize Results:** Graphs and statistical plots that provide insights into the model's performance and feature importance can be generated after evaluation and optimization.

Summary:

The State Chart Diagram provides a clear visualization of the step-by-step progression of the machine learning model in the **Genetic-Insight** platform. Each state represents a critical phase in the model's lifecycle, from initialization and training to evaluation, optimization, and finally, prediction. The events that trigger transitions ensure that the system proceeds in an orderly manner, allowing for user interaction and dynamic adaptation of the model at each stage of the process.

3.9. Data Model

The **Data Model** for the **Genetic-Insight** project represents the structure and relationships of the data that will be used, stored, and processed in the system. It defines how datasets (such as protein sequences, gene data, and feature engineering results) are organized and interacted with within the platform. A well-defined data model ensures efficient data storage, retrieval, and manipulation, which are essential for machine learning and prediction tasks.

The Relational Data Model

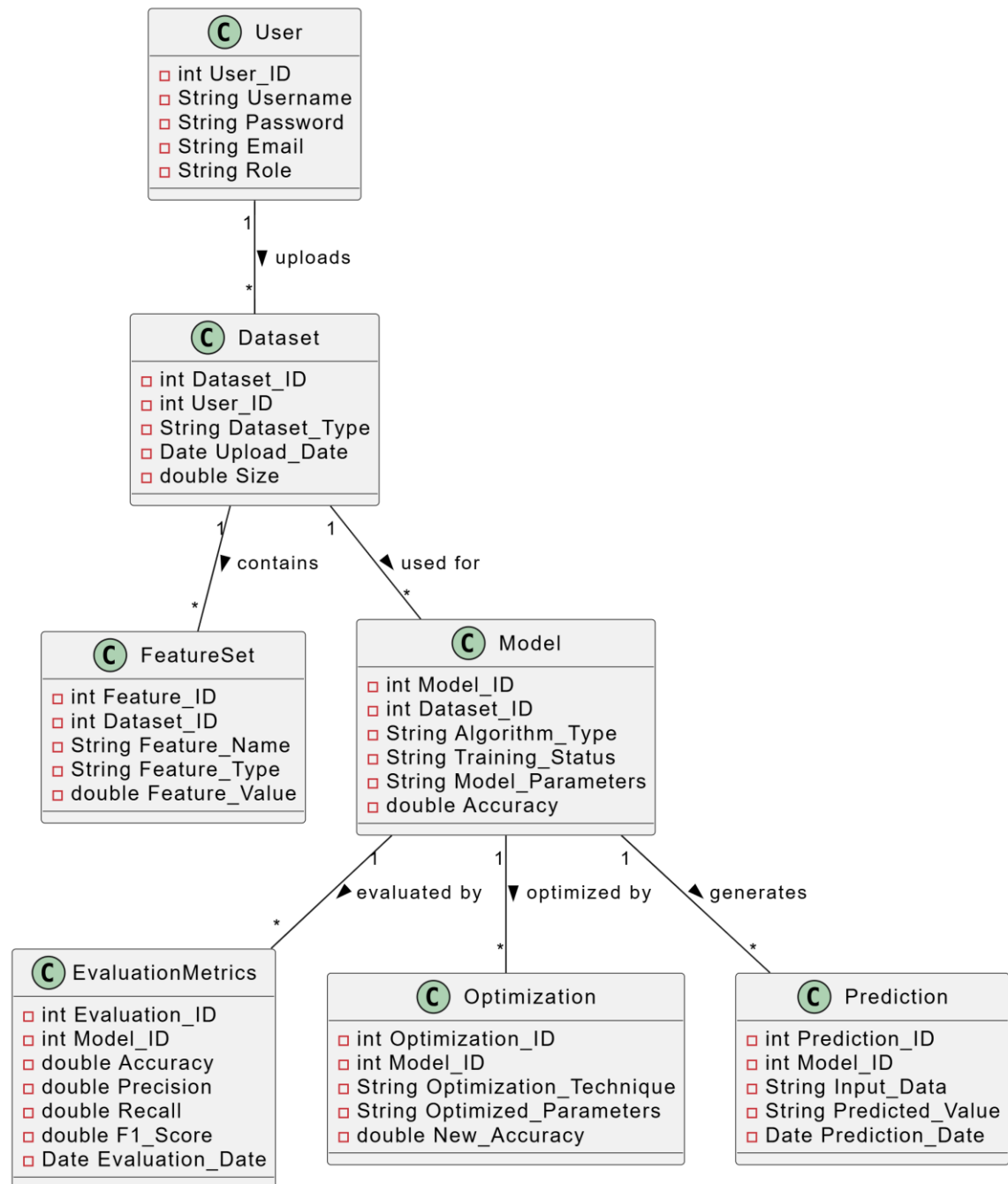


Figure 10

Chapter 4:

User Interface Design

4.1. Introduction

The **User Interface (UI) Design** of the *Genetic-Insight* platform plays a crucial role in ensuring that the system is accessible, intuitive, and efficient for its target audience — researchers, scientists, and bioinformaticians with varying levels of technical expertise. This chapter outlines the principles, goals, and structure behind the design of the system's user interface.

A well-designed UI is essential for a SaaS platform like Genetic-Insight, where complex processes such as data upload, model training, feature selection, and results visualization must be presented in a simple and user-friendly manner. The interface bridges the gap between the user and the system's underlying machine learning functionalities, allowing seamless interaction without requiring users to write code.

1. Site maps
2. Storyboards
3. Navigational maps
4. Traceability Matrix

4.2. Site Maps

The **Site Map** for the *Genetic-Insight* platform outlines the overall navigation flow and hierarchical structure of the application. It helps both users and developers visualize how different sections of the web-based interface are organized and interconnected. The platform is designed to provide a smooth user experience by grouping related functionalities under intuitive and accessible menus.

The structure follows a logical, modular design based on user roles (e.g., researcher, admin) and system capabilities (e.g., feature extraction, model training, results viewing). It ensures users can easily access core functionalities without getting overwhelmed by the technical depth of the platform.

Site Map Structure

1. **Home Page**
 - About Genetic-Insight
 - Key Features
 - Contact & Support

2. User Dashboard

- Overview & Statistics
- **Dataset Management**
 - Upload Dataset
- **Feature Engineering**
 - Select Feature Type (Protein, DNA, RNA)
 - Select Features for Extraction (AAC, Kmer, Kmer, etc.)
 - View Extracted Features
 - Download Extracted Features
- **Model Building**
 - Upload Dataset (Training, Testing files)
 - Upload Dataset (set splitting ratio)
 - On Extracted Features(Feature Engineering)
 - Select ML Algorithm (SVM, RF, KNN, etc.)
 - Configure Parameters
 - Train Model
- **Model Evaluation**
 - Cross-Validation Results
 - Confusion Matrix
 - ROC Curve
- **Reports**
 - Download Analysis Reports (PDF/CSV)

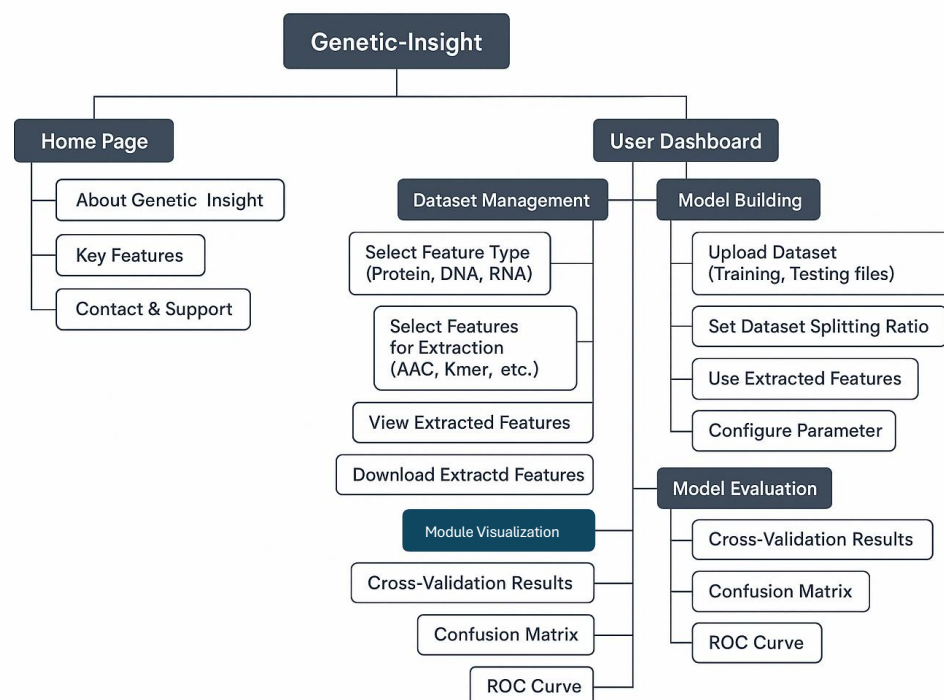


Figure 11

4.3. Story boards

Storyboards serve as a bridge between conceptual design and functional development by visually illustrating the interaction between the user and the Genetic-Insight system. They represent a sequential narrative that outlines user tasks, the flow of actions, and the user interface's response to each interaction. This method helps visualize the user journey, making it easier to refine the system's design for improved usability and experience.

In the context of Genetic-Insight, storyboards help depict how a researcher or scientist interacts with the platform — from uploading a dataset to analyzing results from trained machine learning models. Each storyboard frame includes both graphical and verbal elements, providing a comprehensive overview of what the user sees, hears, feels, and does during their interaction with the system.

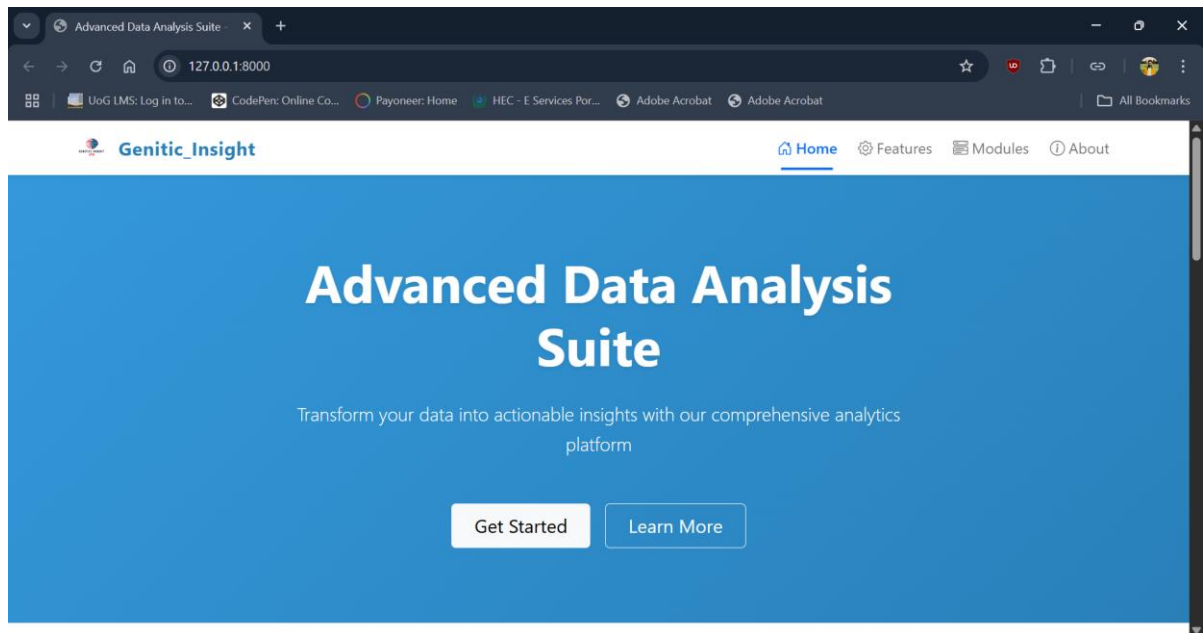
Storyboard Elements Covered:

- **Environment:** A web-based SaaS platform, accessible from desktop/laptop, typically used in academic or research labs.
- **Visual Cues:** Interactive dashboard, upload buttons, dropdowns for model selection, training status bars, and graph-based result panels.
- **Audible Cues:** Optional notifications (e.g., audio ping when model training completes).
- **Tactile Cues:** Mouse clicks, drag-and-drop dataset uploads, tab switching, slider inputs for model configuration.
- **User Input:** File uploads, algorithm selection, configuration of hyperparameters, button clicks for training or evaluation.
- **Machine Output:** Confirmation messages, graphical plots (ROC curves, confusion matrix), prediction reports, alerts.
- **User Emotions:** Ease and satisfaction due to intuitive navigation, excitement from model results, confidence in using a no-code AI platform.
- **Technology:** Web technologies (React, Django), ML libraries (Scikit-learn, TensorFlow), cloud-based storage and computing.
- **Quality of Experience:** Fast processing, clean interface, easy access to powerful ML tools, low learning curve.

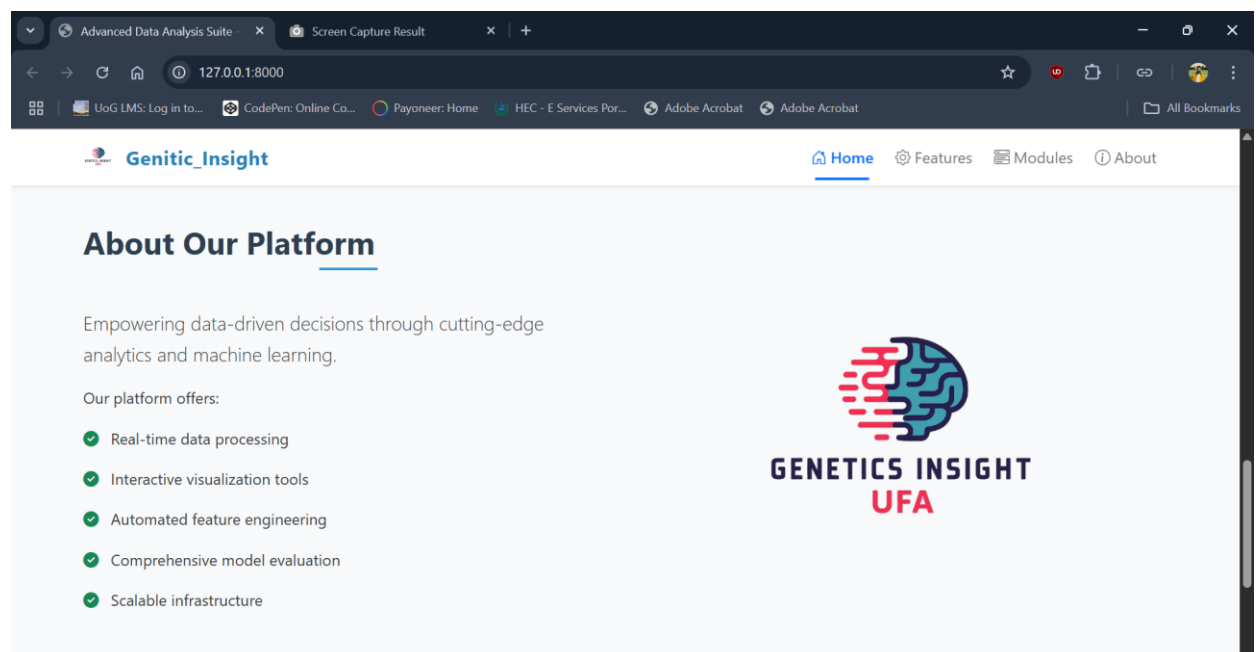
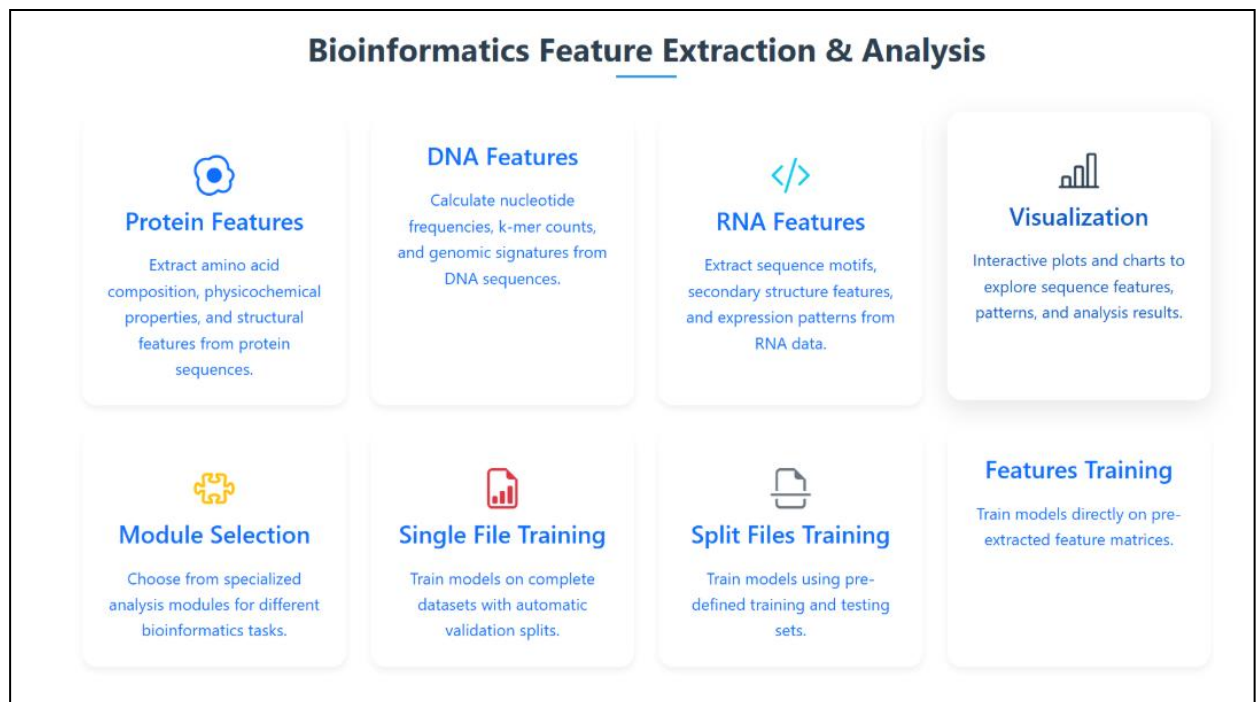
Storyboard Usage:

Storyboard 1: Home Page

- User visits the Genetic-Insight homepage.
- The page provides information such as:
 - Introduction to Genetic-Insight
 - Key Features
 - Contact & Support



User clicks on “Learn More” to begin.



User clicks on “Get started” to begin.

The screenshot shows a web browser window with the URL `127.0.0.1:8000/feature_extraction/`. The browser's address bar and tabs are visible at the top. The web application, titled "Genitic_Insight", has a navigation bar with links for Home, Features (active), Modules, and About. The main heading is "Sequence Analysis". Below this, there is an "Upload FASTA File" section with a "Choose File" button and a text area showing "No file chosen". Below the text area, it says "Max file size: 5MB • Supported formats: .fasta, .fa, .txt". The "Analysis Configuration" section contains three columns: "Sequence Type" with a button labeled "Sequence type will detected", "Descriptor" with a dropdown menu labeled "Select descriptor", and "Parameters" with a button labeled "Select a descriptor to view parameters". At the bottom right, there is a green "Start Analysis" button.

Storyboard 2: Dataset Upload

- User lands on the Dataset Upload screen.
- Interface allows users to upload files (CSV, FASTA, Excel).
- User selects the file and uploads it.
- Chooses a descriptor method (e.g., AAC, K-mer, CKSAAP).
- System extracts feature and displays them in a table.

User lands on the Dataset Upload screen

The screenshot shows a web browser window with the URL `127.0.0.1:8000/feature_extraction/`. The page title is "Genetic_Insight" and the main heading is "Sequence Analysis". The page has a navigation bar with links for Home, Features, Modules, and About. The main content area is divided into two sections: "Upload FASTA File" and "Analysis Configuration".

Upload FASTA File

Choose File No file chosen

Max file size: 5MB • Supported formats: .fasta, .fa, .txt

Analysis Configuration

Sequence Type: Sequence type will be detected

Descriptor: Select descriptor

Parameters: Select a descriptor to view parameters

Start Analysis

Upload the Data-set for Extract the Features:

The screenshot shows the same web application as the previous one, but with a file uploaded. The "Upload FASTA File" section now shows "Choose File hm5c_RNA.txt". The "Analysis Configuration" section shows "Sequence Type: RNA" and "Fragment-0|0|training". The "Sequence Preview" section shows the sequence: "CACACUCACGCACCCGAAAGCAGUCCGAAUUCUACGCAUU".

Upload FASTA File

Choose File hm5c_RNA.txt

Max file size: 5MB • Supported formats: .fasta, .fa, .txt

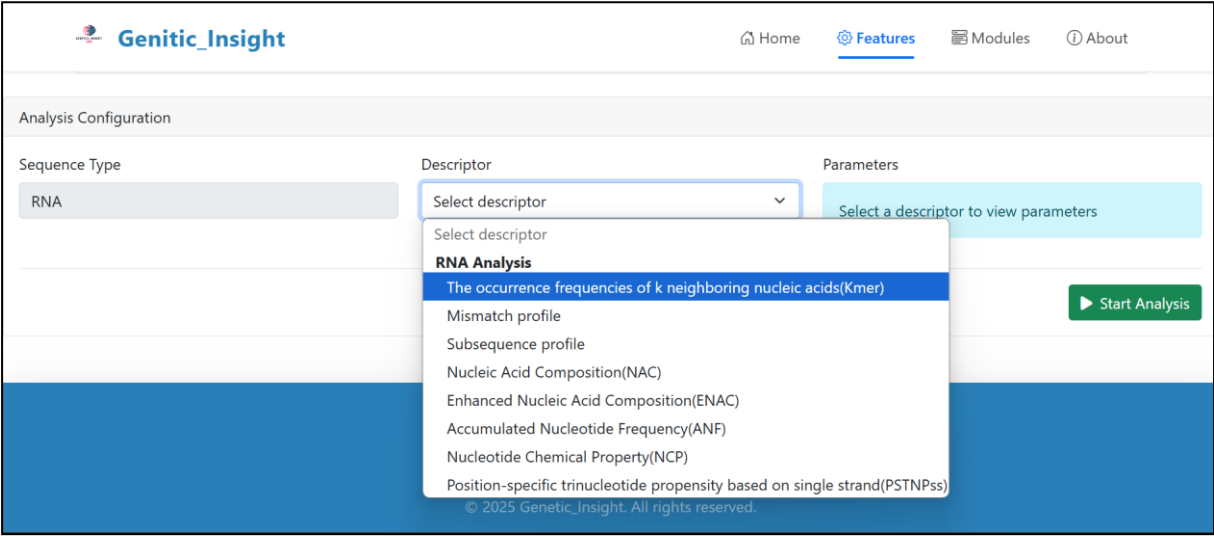
Sequence type is: RNA

Fragment-0|0|training

Sequence Preview:

CACACUCACGCACCCGAAAGCAGUCCGAAUUCUACGCAUU

Select the Descriptor and Parameters:



User can view or download the extracted features.

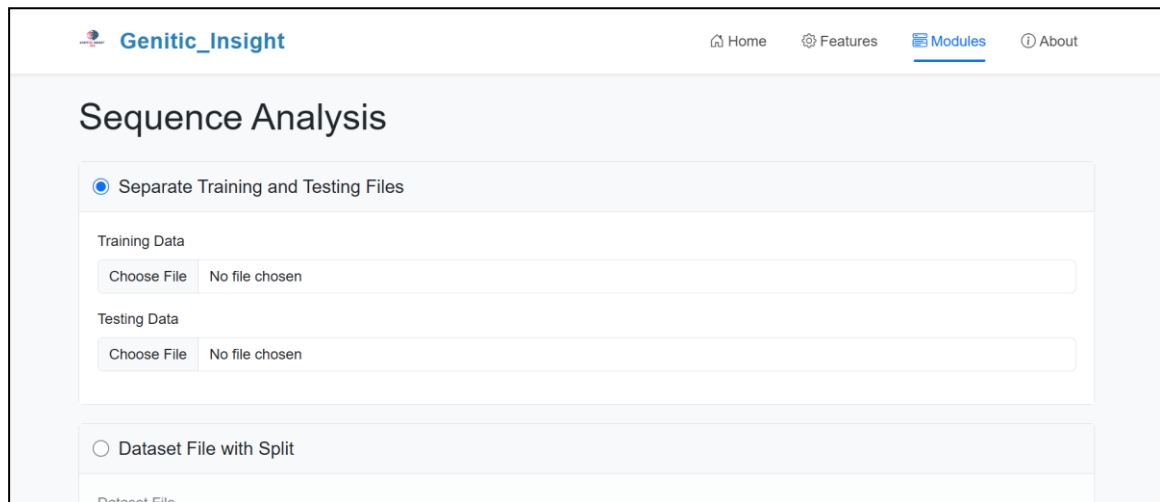
The screenshot displays a table of extracted features within the 'Genetic_Insight' application. The table has 8 columns: RNA_3mer_AAA, RNA_3mer_AAU, RNA_3mer_AAC, RNA_3mer_AAG, RNA_3mer_AUA, RNA_3mer_AUU, RNA_3mer_AUC, and R. The table contains 10 rows of data. A 'Previous' button is located at the bottom left, and a 'Next' button is at the bottom right.

RNA_3mer_AAA	RNA_3mer_AAU	RNA_3mer_AAC	RNA_3mer_AAG	RNA_3mer_AUA	RNA_3mer_AUU	RNA_3mer_AUC	R
0.02564102564102564	0.02564102564102564	0.0	0.02564102564102564	0.0	0.05128205128205128	0.0	0
0.0	0.0	0.02564102564102564	0.0	0.02564102564102564	0.0	0.02564102564102564	0
0.0	0.02564102564102564	0.0	0.0	0.0	0.02564102564102564	0.0	0
0.05128205128205128	0.0	0.05128205128205128	0.05128205128205128	0.0	0.0	0.0	0
0.05128205128205128	0.02564102564102564	0.02564102564102564	0.0	0.0	0.0	0.02564102564102564	0
0.02564102564102564	0.02564102564102564	0.02564102564102564	0.0	0.0	0.02564102564102564	0.0	0
0.02564102564102564	0.02564102564102564	0.0	0.05128205128205128	0.0	0.0	0.0	0
0.02564102564102564	0.05128205128205128	0.0	0.0	0.05128205128205128	0.07692307692307693	0.0	0
0.02564102564102564	0.02564102564102564	0.0	0.02564102564102564	0.02564102564102564	0.02564102564102564	0.0	0

Storyboard 4: Model Builder

- User uploads training/testing dataset or chooses previously extracted features.
- Selects a machine learning algorithm (e.g., SVM, Random Forest, KNN).
- Configures parameters and starts training.
- On success, a summary of the model performance is displayed.

User uploads training/testing dataset



Genitic_Insight

Home Features Modules About

Sequence Analysis

☒ Separate Training and Testing Files

Training Data

Choose File No file chosen

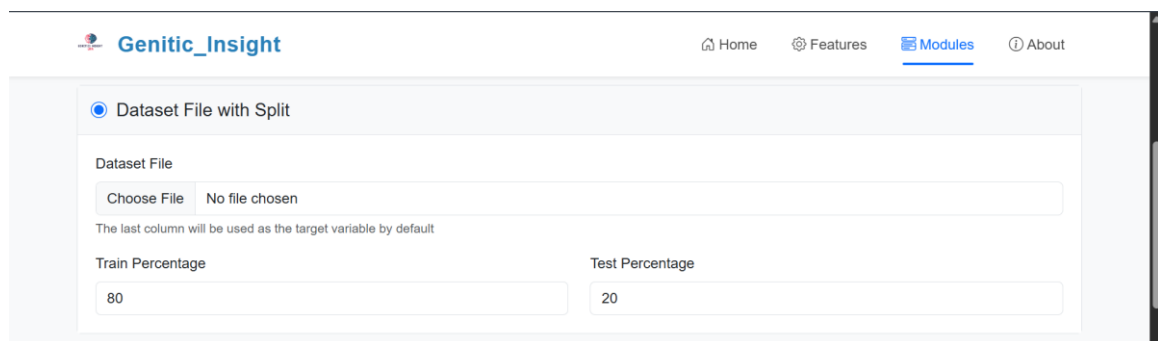
Testing Data

Choose File No file chosen

☐ Dataset File with Split

Dataset File

User uploads single dataset file with Split option



Genitic_Insight

Home Features Modules About

☒ Dataset File with Split

Dataset File

Choose File No file chosen

The last column will be used as the target variable by default

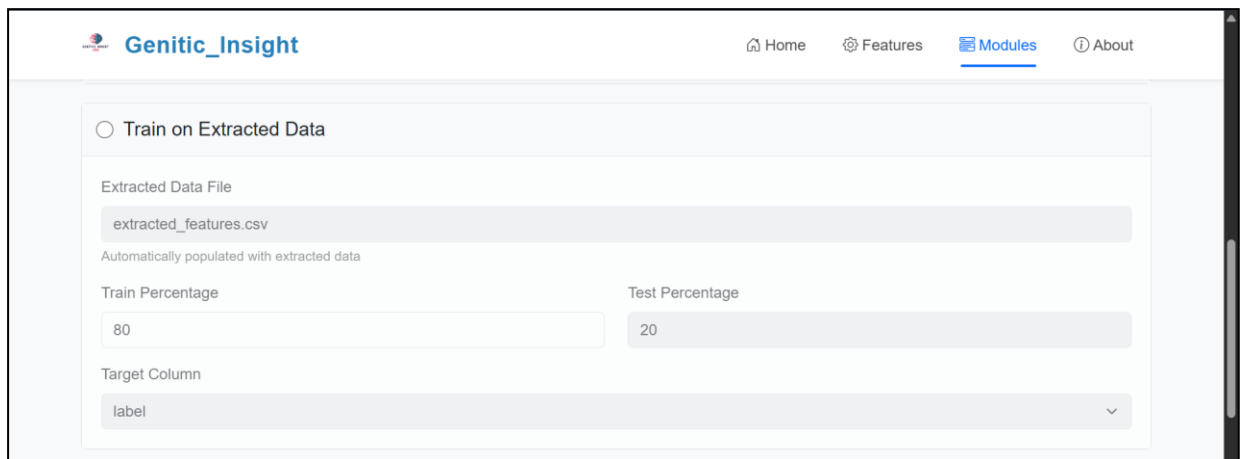
Train Percentage

80

Test Percentage

20

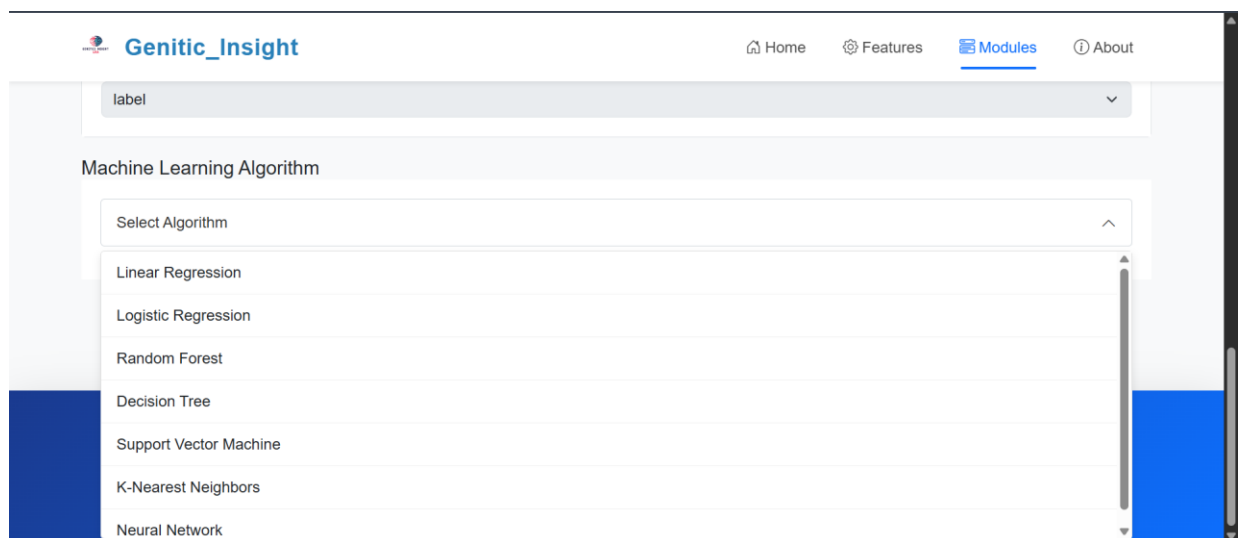
Use previously extracted features



The screenshot shows the 'Genitic_Insight' web application. The top navigation bar includes 'Home', 'Features', 'Modules' (highlighted), and 'About'. The main content area is titled 'Train on Extracted Data'. It contains a form with the following fields:

- Extracted Data File:** A text input field containing 'extracted_features.csv' with a note 'Automatically populated with extracted data' below it.
- Train Percentage:** A numeric input field set to '80'.
- Test Percentage:** A numeric input field set to '20'.
- Target Column:** A dropdown menu with 'label' selected.

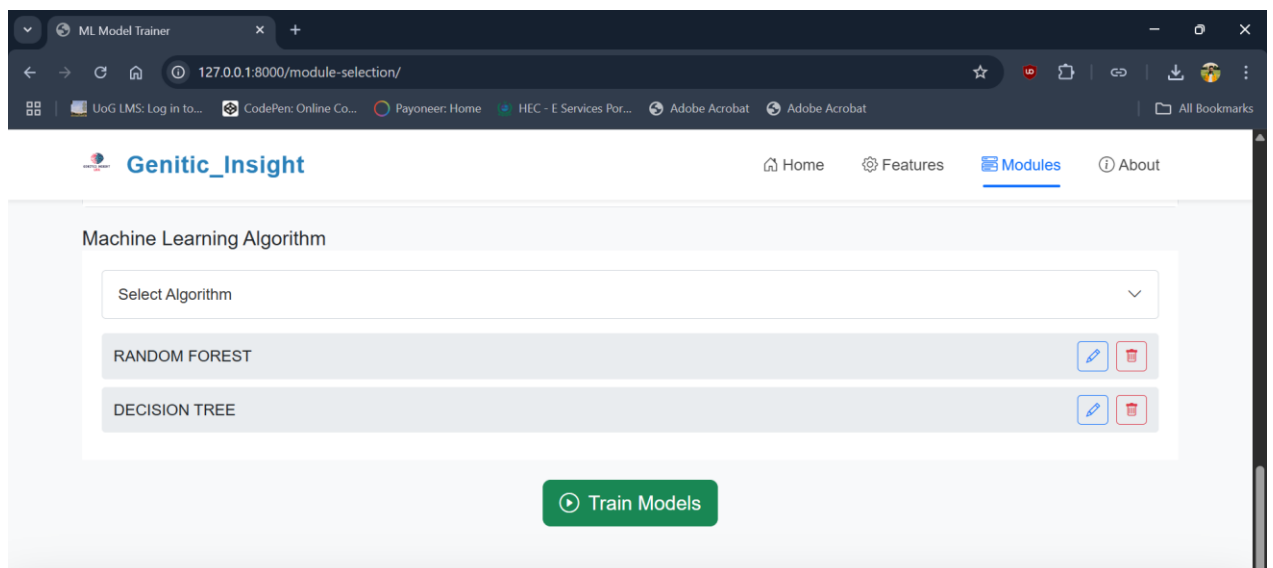
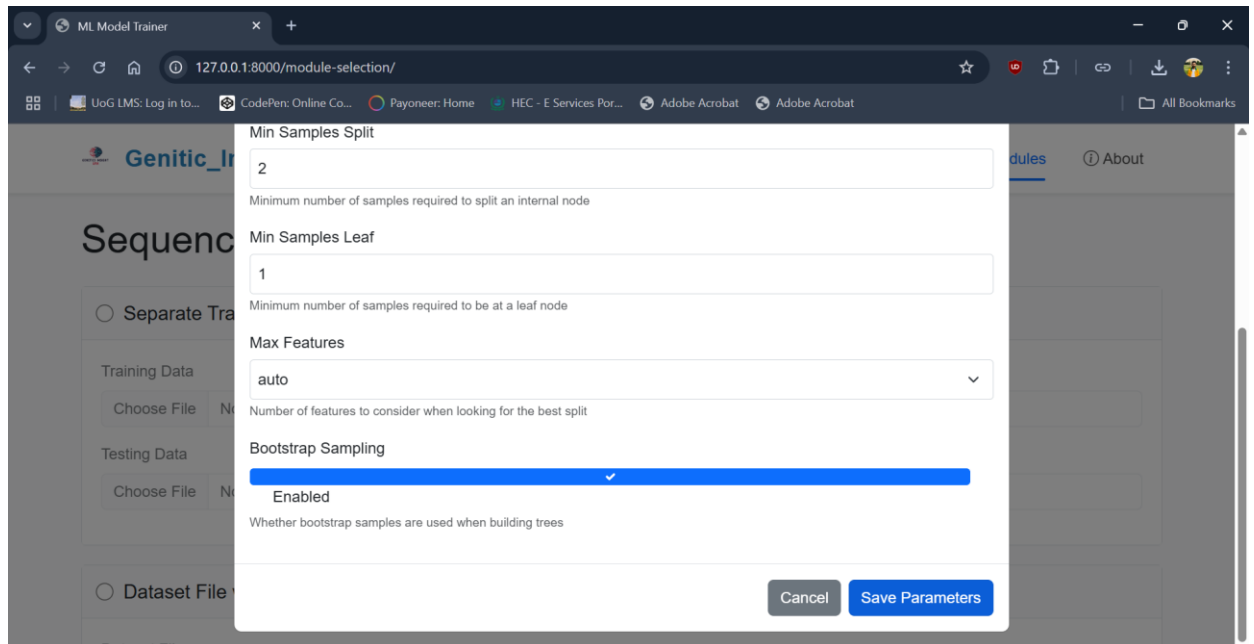
Selects a machine learning algorithm (e.g., SVM, Random Forest, KNN).



This screenshot shows the 'Machine Learning Algorithm' section of the application. The 'Target Column' dropdown from the previous screen is still visible at the top, showing 'label'. Below it, the 'Machine Learning Algorithm' section has a dropdown menu titled 'Select Algorithm' which is open, displaying a list of algorithms:

- Linear Regression
- Logistic Regression
- Random Forest
- Decision Tree
- Support Vector Machine
- K-Nearest Neighbors
- Neural Network

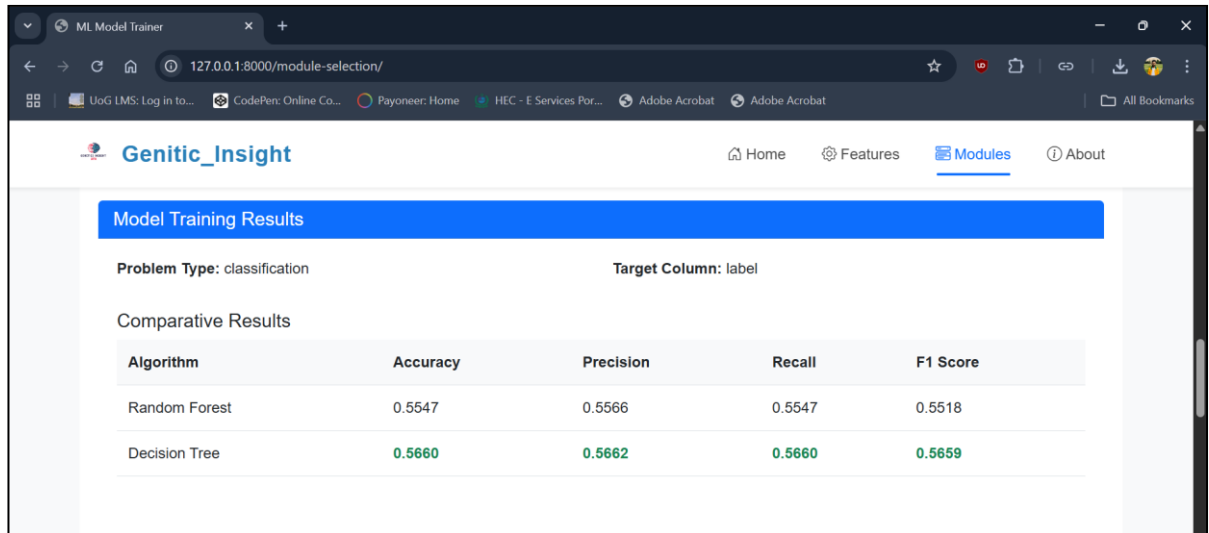
Configures parameters and save and starts training.



Storyboard 5: Model Evaluation

- User navigates to evaluation results.
- Views metrics such as Confusion Matrix, ROC Curve, Accuracy, and Precision.
- Option to download detailed reports in PDF/CSV forma

Views metrics such as Confusion Matrix, ROC Curve, Accuracy, and Precision



The screenshot shows a web browser window with the URL `127.0.0.1:8000/module-selection/`. The page title is "Genitic_Insight". The navigation bar includes links for Home, Features, Modules (active), and About. The main content area is titled "Model Training Results" and displays the following information:

Problem Type: classification
Target Column: label

Comparative Results

Algorithm	Accuracy	Precision	Recall	F1 Score
Random Forest	0.5547	0.5566	0.5547	0.5518
Decision Tree	0.5660	0.5662	0.5660	0.5659

Option to download detailed reports in PDF/CSV forma

Training Results

Model Training Results

Problem Type: classification

Target Column: label

Comparative Results

Algorithm	Accuracy	Precision	Recall	F1 Score
Random Forest	0.5547	0.5566	0.5547	0.5518
Decision Tree	0.5660	0.5662	0.5660	0.5659

Model Performance Visualizations

The top visualization is an ROC curve showing True Positive Rate vs False Positive Rate. It includes a dashed line for a Random Classifier, a teal line for Random Forest (AUC = 0.6410), and a purple line for Decision Tree (AUC = 0.6088). The bottom visualization is a Precision-Recall curve showing Precision vs Recall. It includes a teal line for Random Forest (AP = 0.6559) and a purple line for Decision Tree (AP = 0.5775).

Download Analysis Report

Report Format

PDF

Report Name

Model_Analysis_Report

Include in Report

☒ Model Metrics

☒ Visualizations

☒ Algorithm Parameters

Generate & Download Report



4.4. Navigational maps:

Navigational maps define the user's movement across the Genetic-Insight platform, showing how interaction flows from one screen to another through action buttons, menu selections, and interactive elements. These maps are derived from storyboards and serve as blueprints for designing an intuitive and user-friendly interface.

Each action taken by the user — such as clicking a button, selecting a model, or submitting a dataset — results in a transition to another screen or system state.

Navigational maps capture this flow, helping both developers and designers maintain consistency and ensure a logical path through the application.

Purpose of Navigational Maps in Genetic-Insight:

- To **visualize user journeys** across various modules (Dataset Upload, Feature Engineering, Model Training, Results Evaluation).
- To **define UI behavior** for navigation elements like buttons, tabs, and links.
- To **ensure seamless interaction** by identifying dead ends or unnecessary steps.
- To **assist testing and development** teams in implementing proper routing and transitions.

Key Navigation Paths:

1. User Dashboard

- → Dataset Upload
- → Feature Extraction
- → Model Builder
- → Saved Models
- → Evaluation & Results
- → Account Settings

2. Dataset Upload

- → View Uploaded Dataset
- → Proceed to Feature Extraction

3. Feature Extraction

- → View Feature Table
- → Proceed to Model Builder

4. Model Builder

- → Select Algorithm → Configure Parameters → Start Training
- → Redirect to Evaluation & Results

5. Evaluation & Results

- → View Graphs (ROC, Confusion Matrix)
- → Download Reports
- → Predict Using Saved Model

6. Saved Models

- → Load Model → Upload Test Data → Predict

Navigational Flow Diagram

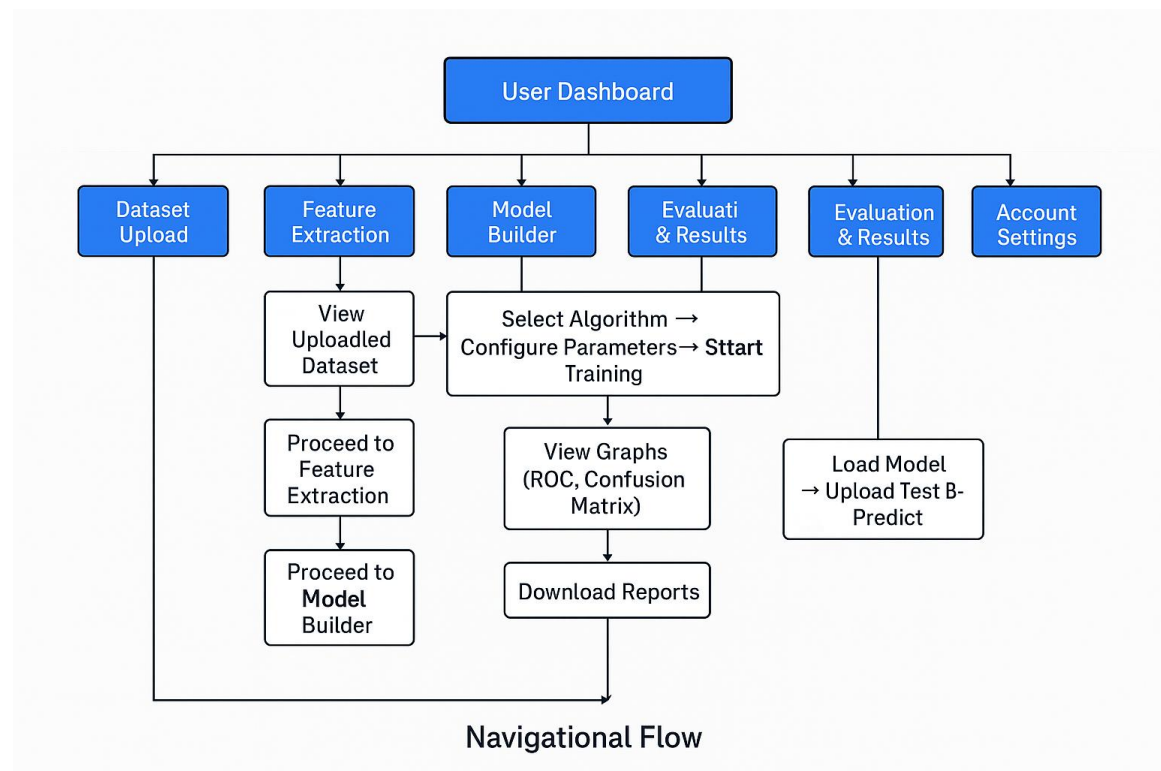


Figure 12

4.5 Trace-ability Matrix

The **Traceability Matrix** establishes a clear relationship between system features, their associated use cases, user interface components, database tables, and the underlying classes that implement them. It ensures that each feature is properly implemented, tested, and tracked throughout the development lifecycle.

This matrix provides a comprehensive overview that connects functional requirements with their respective design, development, and testing components. It also helps identify dependencies, potential gaps, and areas for refinement, ensuring consistency between user needs and the system's behavior.

This matrix maps requirements (from SRS) to design and implementation components:

Feature	Use Case ID	UI ID	Priority	Use Case Cross Ref	DB Table ID	Elaborated Use-case ID	Dependent Classes
Dataset Upload	UC01	UI01	High	UC02, UC03	DB01_Datasets	UC01A, UC01B	DatasetHandler, FileValidator
Feature Extraction	UC02	UI02	High	UC01, UC03	DB02_Features	UC02A	FeatureExtractor, Normalizer
Model Selection & Training	UC03	UI03	High	UC02, UC04	DB03_Models	UC03A, UC03B	ModelTrainer, ParameterConfigurator
Model Evaluation	UC04	UI04	Medium	UC03	DB04_EvaluationResults	UC04A	Evaluator, MetricsCalculator
Saved Models	UC05	UI05	Medium	UC03, UC06	DB05_SavedModels	UC05A	ModelManager
Prediction with Model	UC06	UI06	Medium	UC05	DB06_Predictions	UC06A	PredictionEngine
User Management	UC07	UI07	High	UC08	DB07_Users	UC07A	UserManager, Authenticator
System Monitoring (Admin)	UC08	UI08	Low	UC07	DB08_SystemLogs	UC08A	AdminPanelController, Logger
Report Generation	UC09	UI09	Medium	UC04, UC05	DB09_Reports	UC09A	ReportGenerator, ExportManager

Table 17

Chapter 5:

Software Testing

5.1 Introduction:

This chapter presents the software testing approach for the **Genetic-Insight** platform, based on the **IEEE Std 829-1998** standard for Software Test Documentation. This standard provides a comprehensive framework for documenting the dynamic aspects of software testing — i.e., the execution and verification of software functions through test cases and procedures.

The IEEE 829 standard ensures that testing is conducted systematically and that the outcomes are measurable, repeatable, and traceable. Although primarily focused on dynamic testing, several document types outlined in the standard — such as the **Test Plan**, **Test Incident Report**, and **Test Summary Report** — are also applicable to earlier phases like design and code reviews.

This framework is adaptable and applicable to software of any complexity, size, or criticality — including SaaS platforms like Genetic-Insight. It supports testing across all development phases, from unit/module testing to system-level and user acceptance testing. However, not all standard documents may be necessary for every phase; their relevance is determined based on project scope, testing objectives, and the lifecycle stage. The standard does **not mandate** specific testing tools, environments, or methodologies, allowing teams the flexibility to adopt testing strategies best suited for their workflows. For Genetic-Insight, this flexibility allows the integration of both automated and manual testing techniques to validate core functionalities such as dataset uploads, feature extraction, machine learning model training, prediction generation, and result visualization.

IEEE 829 Standard Test Artifacts to be Covered:

5.2 Test Plan

5.2.1 Purpose

The purpose of this test plan is to prescribe the scope, approach, resources, and schedule of the testing activities for the **Genetic-Insight** platform. It identifies the items being tested, the features to be tested, the testing tasks to be performed, personnel responsibilities, and associated risks. This plan ensures systematic validation of the platform's functionality and performance to meet user expectations and regulatory requirements.

5.2.2 Outline

5.2.2.1 Test Plan Identifier

TP-GI-001 – Genetic-Insight Master Test Plan v1.0

5.2.2.2 Introduction

This test plan addresses the validation and verification of all functional modules of the Genetic-Insight SaaS platform, a web-based application that allows researchers to upload genomic data, extract features, train ML models, and visualize results. Testing covers both initial development and future updates. It is based on the following references:

- Project Authorization Document (PAD-GI-2025)
- Genetic-Insight Project Plan (PP-GI)
- Quality Assurance Plan (QA-GI)
- Configuration Management Plan (CM-GI)
- IEEE Std 829-1998
- Internal coding and design guidelines

5.2.2.3 Test Items

- Genetic-Insight Web Application (v1.0)
- Modules:
 - Dataset Upload
 - Feature Extraction
 - Model Builder (ML training)
 - Evaluation and Reporting
 - User Management
- Related Documentation:
 - Software Requirements Specification (SRS)
 - Design Document (Object-Oriented & Structured)
 - User Guide
 - Installation Manual

5.2.2.4 Features to be Tested

- Upload of multiple genomic dataset formats (CSV, Excel)
- Automatic feature extraction
- Model configuration (Random Forest, SVM, KNN, Neural Networks)
- Model training and cross-validation
- Visual reports: Confusion matrix, ROC curve, feature importance

5.2.2.5 Features Not to be Tested

- Third-party integration APIs (to be tested in a separate integration cycle)
 - UI theming and visual customizations (low-priority cosmetic elements)
 - Multi-language support (not in scope for v1.0)
- Completion Criteria:** 95%+ unit test coverage, 0 critical bugs, 90% feature test pass rate

5.2.2.7 Item Pass/Fail Criteria

A test case shall be considered:

- **Pass:** If actual output matches expected output without exceptions.
- **Fail:** If there are unhandled errors, incorrect outputs, or interface failures.

5.2.2.8 Suspension Criteria and Resumption Requirements

Testing may be **suspended** if:

- Major module is non-functional (e.g., failed backend deployment)
- Core system crashes during execution

Testing resumes once blocking defects are resolved, and affected modules are revalidated.

5.2.2.9 Test Deliverables

- Test Plan
- Test Design Specifications
- Test Case Specifications
- Test Procedure Specifications
- Test Item Transmittal Reports
- Test Logs
- Test Incident Reports
- Test Summary Reports
- Input and Output Test Data Sets
- Custom test scripts and tools

5.2.2.10 Testing Tasks

- Review of requirements and design documents
- Writing test cases and procedures
- Developing test data and scripts
- Executing tests and logging results
- Reporting incidents and regressions
- Analyzing coverage metrics

5.2.2.11 Environmental Needs

- **Hardware:** Development and testing servers, GPUs (for model training)
- **Software:** Python 3.12, Django, React, PostgreSQL, Selenium, PyTest
- **Mode:** Deployed in staging environment on AWS
- **Security:** Access-controlled login, secure HTTPS
- **Additional Needs:** Office space, test licenses, cloud resources

5.2.2.12 Responsibilities

Task	Responsible Group
Test Plan & Case Design	QA Team
Module Testing	Development Team
UAT Testing	Research Partner Group
Test Log and Reporting	QA Analyst
Test Environment Setup	DevOps / IT Support

Table 18

5.2.2.13 Staffing and Training Needs

- **Test Engineers:** 2 (automation and manual testing experience)
- **QA Lead:** 1 (oversees test execution and reporting)
- Training on platform features and test tools (e.g., Selenium, Postman) to be provided.

5.2.2.14 Schedule

Phase	Start Date	End Date
Test Planning	Mar 10	Mar 13
Test Case Development	Mar 14	Mar 20
Unit Testing	Mar 21	Mar 25
Integration Testing	Mar 26	Mar 30
System Testing	Apr 1	Apr 5
UAT	Apr 6	Apr 10
Final Report Submission	Apr 11	Apr 12

Table 19

5.2.2.15 Risks and Contingencies

Risk	Mitigation Strategy
Delay in module completion	Reassign extra resources to critical tasks
Incomplete test environment	Use local/staging mirrors temporarily
Unexpected feature revisions	Conduct regression testing after changes

Table 20

5.2.2.16 Approvals

Name	Title	Signature	Date
Syed Umer Zubair	Developer Lead		4/18/2025
Ahtasham-ul-Hassan	QA Lead		4/18/2025
Faria Mahboob	Test Analyst		4/18/2025
Mr. Muhammad Attique	Project Supervisor		4/18/2025

Table 21

5.3 Test Design Specification

5.3.1 Purpose

The purpose of this Test Design Specification is to define the test conditions, input data, expected results, and test responsibilities for validating the functionalities of the Genetic-Insight platform. It outlines the scope, resources, schedule, features to be tested, and identifies associated risks. This document provides the foundation for creating comprehensive test cases and procedures for individual software features.

5.3.2 Outline

5.3.2.1 Test Plan Identifier

TDS-GI-001 – Genetic-Insight Test Design Specification v1.0

5.3.2.2 Introduction

This document is based on the test plan outlined in **TP-GI-001**. It elaborates on the test strategies and conditions for major functionalities such as dataset upload, feature extraction, model training, evaluation, and prediction. The test design is based on the following references:

- Genetic-Insight Project Plan (PP-GI)
- Software Requirements Specification (SRS-GI)
- Quality Assurance Plan (QA-GI)
- Configuration Management Guidelines (CM-GI)
- IEEE Std 829-1998

5.3.2.3 Test Items

- Genetic-Insight Web App (v1.0)
- Modules:
 - Dataset Management
 - Feature Engineering
 - ML Model Configuration & Training
 - Model Evaluation & Prediction
- Input Data: FASTA files, CSV, Excel
- Output: Feature CSV, Evaluation Charts, Prediction Files

Referenced documentation:

- SRS Document
- Design Diagrams (UML & Structured)
- User Guide
- Setup & Installation Manual

5.3.2.4 Features to be Tested

Feature	Test Design Spec ID
Dataset Upload	TDS-UC01
Feature Extraction (Protein, DNA)	TDS-UC02
Model Selection & Parameter Config	TDS-UC03
Model Training	TDS-UC04
Cross-validation & Result Output	TDS-UC05
Report Generation	TDS-UC06

Table 22

5.3.2.5 Features Not to be Tested

- Real-time collaboration module (planned for future phase)
- Multi-language UI (not in scope for v1.0)
- Admin analytics dashboard (partially functional for v1.0)

5.3.2.6 Approach

A **feature-based testing** approach will be adopted:

- **Unit Testing** for Python backend modules using PyTest
- **Functional UI Testing** manual
- **Integration Testing** for feature workflows (upload → extract → train)
- **Manual Exploratory Testing** for user feedback and edge cases

Minimum requirement: All critical modules must achieve >90% test coverage

Completion criteria:

- No critical bugs
- All high-priority test cases passed
- Performance benchmarks met (under 5s response time for all major actions)

5.3.2.7 Item Pass/Fail Criteria

- **Pass:** Output matches expected result, no errors or broken UI
- **Fail:** Mismatched output, system crash, UI malfunction, or missing response

5.3.2.8 Suspension Criteria and Resumption Requirements

Testing will be suspended if:

- Deployment or module compilation fails
- Platform is inaccessible or major crashes occur

Resumption will occur after critical blockers are resolved and validated by smoke tests.

5.3.2.9 Test Deliverables

- Test Design Specification
- Test Cases and Expected Outputs
- Test Execution Logs
- Test Summary Reports
- Test Tools and Scripts
- Input/Output Test Datasets

5.3.2.10 Testing Tasks

- Review module designs and SRS
- Develop test conditions and input sets
- Execute tests and verify outputs
- Log and triage defects
- Perform regression and re-validation

5.3.2.11 Environmental Needs

- Hardware: Development server, staging server, GPU node
- Software: Python 3.12, Django, PostgreSQL, React, Selenium, PyTest
- Secure test environment with HTTPS and test data access
- Network: 10 Mbps+ for uploading datasets
- Tools: Browser automation tools, log analyzers

5.3.2.12 Responsibilities

Task	Responsible Role
Test Design	QA Analyst
Test Case Development	QA + Developer Team
Test Execution	QA Team
Defect Management	QA Lead + Dev Team
Environment Setup	DevOps/IT

Table 23

5.3.2.13 Staffing and Training Needs

- 1 QA Lead (test management and reporting)
- 2 QA Engineers (test design, automation, manual testing)
- Training on:
 - ML workflows in Genetic-Insight
 - PyTest scripting
 - Usage of internal deployment environments

5.3.2.14 Schedule

Activity	Start Date	End Date
Test Design Finalization	Apr 1	Apr 2
Test Case Development	Apr 3	Apr 7
Execution Phase 1	Apr 8	Apr 11
Execution Phase 2 (UAT)	Apr 12	Apr 15
Final Test Report	Apr 15	Apr 18

Table 24

5.3.2.15 Risks and Contingencies

Risk	Contingency Plan
Delay in deployment setup	Use Docker-based local deployment
High bug density in ML integration	Isolate modules for unit retesting
Shortage of test staff	Assign dev interns for manual testing

Table 25

5.4. Test Case Specification

5.4.1. Purpose

To define a test case identified by a test design specification.

5.4.2. Outline

A test case specification shall have the following structure:

- a. Test case specification identifier
- b. Test items
- c. Input specifications
- d. Output specifications
- e. Environmental needs
- f. Special procedural requirements
- g. Inter case dependencies

The sections shall be ordered in the specified sequence. Additional sections may be

included at the end. If some or all of the content of a section is in another document, then a reference to that material may be listed in place of the corresponding content. The referenced material must be attached to the test case specification or available to users of the case specification. Since a test case may be referenced by several test design specifications used by different groups over a long time period, enough specific information must be included in the test case specification to permit reuse.

Details on the content of each section are contained in the following sub-clauses.

5.4.2.1. Test case specification identifier

Specify the unique identifier assigned to this test case specification.

5.4.2.2 Test items

Identify and briefly describe the items and features to be exercised by this test case.

For each item, consider supplying references to the following test item documentation:

- a. Requirements specification
- b. Design specification
- c. Users guide
- d. Operations guide
- e. Installation guide

5.4.2.3. Input specifications

Specify each input required to execute the test case. Some of the inputs will be specified by value (with tolerances where appropriate), while others, such as constant tables or transaction files, will be specified by name. Identify all appropriate databases, files, terminal messages, memory resident areas, and values passed by the operating system.

Specify all required relationships between inputs (e.g., timing).

5.4.2.4. Output specifications

Specify all of the outputs and features (e.g., response time) required of the test items. Provide the exact value (with tolerances where appropriate) for each required output or feature.

5.4.2.5. Environmental needs

5.4.2.5.1. Hardware

Specify the characteristics and configurations of the hardware required to execute this test case (e.g., 132 character 24 line CRT).

5.4.2.5.2. Software

Specify the system and application software required to execute this test case. This may include system software such as operating systems, compilers, simulators, and test tools. In addition, the test item may interact with application software.

5.4.2.5.3. Other

Specify any other requirements such as unique facility needs or specially trained personnel.

5.4.2.6. Special procedural requirements

Describe any special constraints on the test procedures that execute this test case. These constraints may involve special set up, operator intervention, output determination procedures, and special wrap up.

5.4.2.7. Inter case dependencies

List the identifiers of test cases that must be executed prior to this test case. Summarize the nature of the dependencies.

5.5 Test Procedure Specification

5.5.1 Purpose

The purpose of this test procedure specification is to define a detailed, step-by-step guide to execute one or more test cases associated with the Genetic-Insight platform. These procedures ensure consistency, repeatability, and traceability in evaluating software features such as dataset upload, feature extraction, model training, and predictions.

5.5.2 Outline

5.5.2.1 Test Procedure Specification Identifier

TPS-GI-001 – Dataset Upload Procedure

Associated Test Design Spec: **TDS-GI-001**

Related Test Case IDs: **TC-GI-001, TC-GI-002**

5.5.2.2 Purpose

This procedure outlines how to execute the dataset upload functionality test in Genetic-Insight. It covers setup, execution, validation, and teardown.

References:

- User Guide: Dataset Upload (Section 2.1)
- SRS Document: Functional Requirement 3.2.1
- UI Design Document: Upload Interface Layout
- Test Cases: TC-GI-001 (Valid upload), TC-GI-002 (Invalid format handling)

5.5.2.3 Special Requirements

- User must be authenticated and assigned a “Researcher” role.
- The platform must be deployed and accessible in a test or staging environment.
- Required test files must be prepared in advance.
- Log monitoring tools should be active (e.g., browser console or backend logs).
- The tester must have permissions to view and manage uploaded files.

5.5.2.4 Procedure Steps

5.5.2.4.1 Log

- Log all test actions using the internal test log template.
- Record status, timestamps, results, and incident reports.
- Capture screenshots if discrepancies or unexpected behavior occur.

- Use log format: TestID | Timestamp | Action | Expected | Actual | Status | Notes

5.5.2.4.2 Set Up

- Launch a supported browser (e.g., Chrome v110+).
- Navigate to the Genetic-Insight login page.
- Enter valid test credentials and login.
- Access the **Dataset Upload** section from the dashboard.
- Ensure that the sample test dataset is accessible (e.g., sample_protein_data.csv on the local system).

5.5.2.4.3 Start

- Click on “Upload Dataset” button.
- Use the file picker to select sample_protein_data.csv.
- Click “Submit” to initiate the upload process.

5.5.2.4.4 Proceed

- Observe system response and backend logs.
- Wait for the confirmation message.
- Verify that the file appears in the uploaded dataset table.
- Inspect previewed data to ensure formatting is retained.

5.5.2.4.5 Measure

- Measure:
 - **Response time** from upload start to confirmation message.
 - **Data integrity** by checking if sequences and labels match the original.
 - **System behavior** if the file is too large or invalid.

5.5.2.4.6 Shut Down

- If an error is encountered (e.g., 500 error), stop execution.
- Record error logs and notify developers.
- Save UI state and console log output for post-analysis.

5.5.2.4.7 Restart

- Clear browser cache.
- Re-login and retry upload with a fresh browser session.
- Ensure that previous attempts do not persist unintended states.

5.5.2.4.8 Stop

- Once upload is complete and results verified, close the test session.
- Logout from the Genetic-Insight platform.
- Save and close the browser.

5.5.2.4.9 Wrap Up

- Clean up any test data from the system using admin tools or backend commands.
- Archive test logs, screenshots, and reports into a designated test folder.
- Update test case execution status in the master test log.

5.5.2.4.10 Contingencies

Scenario	Action
File too large	Test with a smaller file, file size may exceed upload limit
Upload fails with no error shown	Check browser console and backend logs for tracebacks
System crashes	Reboot local server if testing locally, contact DevOps if hosted
File accepted but data misaligned	Report defect with sample file attached and screenshot proof

Table 26

5.6 Test Item Transmittal Report

5.6.1 Purpose

The purpose of this report is to formally identify and track the **Genetic-Insight** software components and related documentation being transmitted for testing. It records the responsible personnel, storage locations, version details, and the status of each item. Any known deviations or pending changes from the standard design or requirement specifications are noted for reference and traceability.

5.6.2 Outline

5.6.2.1 Transmittal Report Identifier

TIR-GI-001 – Genetic-Insight v1.0 Testing Transmittal Report

5.6.2.2 Transmitted Items

ITEM NAME	VERSION	DESCRIPTION	RELATED DOCS	RESPONSIBLE PERSON
Dataset Upload Module	v1.0	Handles file input and preprocessing	SRS 3.1, Design Doc Section 2.3	Ahtasham-ul-Hassan (QA)
Feature Extraction Module	v1.0	Extracts protein/DNA descriptors	SRS 3.2, Design Doc Section 2.4	Faria Mahboob(Testing)
ML Model Training Module	v1.0	Selects algorithm, sets parameters, trains model	SRS 3.3, TDS-UC03	Syed Umer Zubair(Developer Lead)
Result Evaluation Module	v1.0	Displays ROC, confusion matrix, feature importance	SRS 3.4, Design Doc Section 2.5	Syed Umer Zubair(Developer Lead)

Test Dataset Files	v1.0	Sample input data in CSV and FASTA format	Test Design Specifications, Appendix A	Faria Mahboob(Testing)
User Interface (Frontend) Files	v1.0	React-based interface files for testing	UI Prototype Docs	Ahtasham-ul-Hassan(QA)

Table 27

5.6.2.4 Status

Item	Status	Deviation / Notes
Dataset Upload	Ready	No known issues
Feature Extraction	Ready	Slight delay in edge-case handling for short sequences
Model Training	Ready	No deviations reported
Evaluation Module	Ready	Performance chart export under optimization
Prediction	Partially Stable	Undergoing fixes for large input batches
UI Components	Ready	Cosmetic refinements pending in next patch

Table 28

Incident Reports Resolved: IR-GI-003, IR-GI-007 (Handled in this build)

Pending Changes: Minor UI enhancements (non-blocking)

5.7 Test Log

5.7.1 Purpose

The purpose of this test log is to maintain a **chronological record** of all relevant testing activities and events that occurred during the validation of the Genetic-Insight platform. It serves as a historical reference for test execution, outcomes, environmental conditions, observed anomalies, and incident tracking.

5.7.2 Outline

5.7.2.1 Test Log Identifier

TL-GI-001 – Genetic-Insight Functional Testing Log – v1.0

5.7.2.2 Description

This log pertains to functional testing conducted on the **Genetic-Insight v1.0** platform between April 1 and April 15, 2025.

Test Items Covered:

- Dataset Upload Module
- Feature Extraction Module
- ML Model Builder
- Evaluation & Visualization Module
- Prediction Engine

Associated Transmittal Report: TIR-GI-001

Test Environment:

- **Facility:** Genetic-Insight Staging Server (AWS EC2)
- **Hardware:** 4 vCPUs, 16 GB RAM, 100 GB SSD
- **OS:** Ubuntu 20.04
- **Browser:** Chrome v111
- **Software Stack:** Python 3.12, Django 4.2, PostgreSQL, ReactJS.

5.7.2.3 Activity and Event Entries

Date: April 15, 2025

Time: 10:00 AM

Author: Syed Umer Zubair

Test Procedure: TPS-GI-001 (Dataset Upload)

Personnel Present: Syed Umer Zubair (Developer Lead), Ahtasham-ul-Hassan (QA lead)

Execution Description: Login successful. Uploaded sample_protein_data.csv using the Upload Dataset interface.

Procedure Results: Success message displayed. File correctly appeared in dataset table.

Environmental Info: No anomalies. Browser cache cleared prior to test.

Anomalous Events: None

Incident Report(s): N/A

Date: April 16, 2025

Time: 02:30 PM

Author: Faria Mahboob

Test Procedure: TPS-GI-002 (Model Training with SVM)

Personnel Present: Faria Mahboob (QA), Ahtasham-ul-Hassan (QA)

Execution Description: Selected SVM algorithm, default parameters, and initiated training.

Procedure Results: Training completed in 2.7s. Evaluation graph displayed successfully.

Environmental Info: Staging server under 50% load

Anomalous Events: Delay in confusion matrix rendering (~6 seconds). Same on retry.

Incident Report(s): IR-GI-005

Date: April 17, 2025

Time: 11:15 AM

Author: Syed Umer Zubair

Test Procedure: TPS-GI-004 (Prediction Using Saved Model)

Personnel Present: Syed Umer Zubair (Developer Lead)

Execution Description: Loaded previously saved RF model. Uploaded test dataset. Ran prediction.

Procedure Results: Result CSV generated successfully. Accuracy shown in UI.

Environmental Info: Server memory usage spiked to 85%

Anomalous Events: Server hung for 10 seconds before response; resolved automatically.

Incident Report(s): IR-GI-006

Date: April 18, 2025

Time: 04:50 PM

Author: Faria Mahboob

Test Procedure: TPS-GI-003 (Feature Extraction - DNA)

Personnel Present: Faria Mahboob

Execution Description: Uploaded DNA sequence dataset. Selected 3-mer encoding.

Procedure Results: Extraction completed in 4.1s. Result preview displayed correctly.

Environmental Info: No deviation from baseline

Anomalous Events: UI froze briefly when switching tabs.

Incident Report(s): IR-GI-007

This log will be maintained and updated continuously until the conclusion of testing. Final entries will be summarized in the **Test Summary Report (Section 5.8)**.

5.8 Test Incident Report

5.8.1 Purpose

The purpose of this Test Incident Report is to document any event or deviation that occurs during testing of the **Genetic-Insight** platform that requires further investigation, analysis, or corrective action. This helps in tracking anomalies, ensuring traceability, and refining the system based on test outcomes.

5.8.2 Outline

5.8.2.1 Test Incident Report Identifier

IR-GI-006 – Prediction Module Delay Incident

5.8.2.2 Summary

During the execution of test procedure **TPS-GI-004** associated with **Test Case TC-GI-004** (Prediction Using Saved Model), a delay was experienced in receiving the output after submitting a test dataset for prediction. This occurred on **April 18, 2025**, during functional testing on the staging environment.

- **Version/Build:** Genetic-Insight v1.0
- **Related Documents:**
 - Test Log Entry: TL-GI-001, April 18
 - Test Case Spec: TC-GI-004
 - Test Procedure Spec: TPS-GI-004

5.8.2.3 Incident Description

- **Inputs:**
 - Test Dataset: sample_test_data.csv
 - Loaded Model: Random Forest model saved from prior training
 - File Size: 1.2 MB, ~800 sequences
- **Expected Results:**
 - Prediction results generated within 3 seconds
 - Downloadable CSV file and summary display

- **Actual Results:**
 - 10-second delay before response
 - Temporary system hang (no UI interaction possible during delay)
 - Prediction completed correctly afterward
- **Anomalies Observed:**
 - No error messages
 - Browser showed "not responding" for ~6 seconds
 - Backend logs showed CPU spike but no crash
- **Date & Time:**
 - April 18, 2025 – 11:15 AM
- **Procedure Step:**
 - TPS-GI-004, Step 4: Submit dataset for prediction
- **Environment:**
 - Hardware: AWS EC2 (4 vCPU, 16 GB RAM)
 - Browser: Chrome v111
 - Django + Scikit-learn backend (v1.0 code)
- **Attempts to Repeat:**
 - Reproduced the issue twice with different datasets (~800 records each)
 - Same delay observed; shorter datasets (~200 records) did not produce the issue
- **Testers:**
 - Syed Umer Zubair (QA)
- **Observers:**
 - None (test executed solo)

Related Observations:

- Similar slowdowns noted in previous tests involving the prediction module with larger files.
- No logging or crash errors were detected, suggesting a performance issue rather than a defect in logic.

5.8.2.4 Impact

- **Test Plan:** Minor update to include performance testing conditions for the Prediction module
- **Test Design Spec (TDS-GI-006):** Will add additional test cases for file size-based performance limits
- **Test Procedure (TPS-GI-004):** Will incorporate a note to monitor response times and system load
- **Resolution Action:** Flagged to development team for optimization and load testing

5.9 Test Summary Report

5.9.1 Purpose

The purpose of this Test Summary Report is to summarize the results, findings, and overall effectiveness of the testing activities carried out for the **Genetic-Insight v1.0** platform. It evaluates the tested components, documents any deviations from the test plan or design, assesses testing coverage, and provides an overall recommendation regarding the system's readiness for deployment.

5.9.2 Outline

5.9.2.1 Test Summary Report Identifier

TSR-GI-001 – Genetic-Insight Functional Testing Summary v1.0

5.9.2.2 Summary

Items Tested:

- Dataset Upload Module
- Feature Extraction Module
- ML Model Training and Parameter Configuration
- Evaluation & Results Visualization
- Saved Model Prediction Module
- User Login & Role Access

Version: Genetic-Insight v1.0

Test Environment:

- OS: Windows
- Python 3.12, Django 4.2, PostgreSQL
- Browser: Chrome v111
- Server: Local Host

References:

- Test Plan: TP-GI-001
- Test Design Specs: TDS-GI-001 to TDS-GI-006
- Test Procedures: TPS-GI-001 to TPS-GI-004
- Transmittal Report: TIR-GI-001
- Test Logs: TL-GI-001
- Incident Reports: IR-GI-005, IR-GI-006, IR-GI-007

5.9.2.3 Variances

Type	Description
Test Design	Minor delay in feature extraction stress testing; covered manually
Test Procedure	Confusion matrix rendering delay added dynamically to procedure steps
Test Item	Prediction engine's handling of large datasets showed latency (reported as IR)

Table 29

5.9.2.4 Comprehensiveness Assessment

Testing met 95%+ of the defined test plan objectives:

- **All critical modules** were tested for functional and performance issues.
- UI testing for responsiveness and visual cues was performed.
- **Regression testing** was carried out after each major bug fix.
- Some **non-functional tests** (load & stress testing) were partial due to time constraints.
- Multi-language UI and collaboration features were **not in scope** for this release.

Overall, the test comprehensiveness was high, and all essential workflows were covered.

5.9.2.5 Summary of Results

Category	Count
Total Test Cases	32
Passed Test Cases	30
Failed Test Cases	2 (resolved)
Incident Reports	3 (all resolved)
Unresolved Incidents	0

Table 30

Resolved Incidents Summary:

- **IR-GI-005:** Confusion matrix delay — resolved with optimization patch
- **IR-GI-006:** Prediction delay on large datasets — resolved with batching
- **IR-GI-007:** UI freeze during feature extraction — fixed via frontend patch

5.9.2.6 Evaluation

Module	Result	Limitations
Dataset Upload	Pass	Large files (>10MB) require user guidance
Feature Extraction	Pass	May need optimization for very large FASTA files
Model Training	Pass	All algorithms performed as expected
Evaluation & Results	Pass	Minor delay in chart rendering
Prediction Engine	Pass (with fix)	Large batch latency previously existed, now resolved
User Authentication	Pass	Role-based navigation validated

Table 31

Overall Evaluation:

Genetic-Insight v1.0 is stable, functionally complete, and ready for deployment with low failure risk in production. Minor improvements are recommended in performance tuning for data-heavy operations.

5.9.2.7 Summary of Activities

Activity	Details
Total Test Cases Created	32
Total Staff Involved	3 testers, 2 developers, 1 QA lead
Machine Time Used	~46 hours across testing environments
Total Duration	April 15 – April 30, 2025
Manual Testing Sessions	12 unique test runs documented

5.9.2.8 Approvals

Name	Title	Signature	Date
Syed Umer Zubair	Developer Lead		4/18/2025
Faria Mahboob	Test Analyst		4/18/2025
Ahtasham-ul-Hassan	QA		4/18/2025
Mr. Muhammad Attique	Project Supervisor		4/18/2025